Machine Learning

Unit 5

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

What is Learning?

“Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task (or tasks drawn from the same population) more effectively the next time.” --Herbert Simon

"Learning is constructing or modifying representations of what is being experienced." -- Ryszard Michalski

notes.arpanadhikari7.com.np

"Learning is making useful changes in our minds." --Marvin Minsky

Types of Learning

The strategies for learning can be classified according to the amount of inference the system has to perform on its training data. In increasing order we have

1. Rote learning – the new knowledge is implanted directly with no inference at all, e.g. simple memorisation of past events, or a knowledge engineer's direct programming of rules elicited from a human expert into an expert system.

2. Supervised learning – the system is supplied with a set of training examples consisting of inputs and corresponding outputs, and is required to discover the relation or mapping between then, e.g. as a series of rules, or a neural network.

3. Unsupervised learning – the system is supplied with a set of training examples consisting only of inputs and is required to discover for itself what appropriate outputs should be, e.g. a *Kohonen Network* or *Self Organizing Map*.

Early expert systems relied on rote learning, but for modern AI systems we are generally interested in the supervised learning of various levels of rules.

Available on notes.arpanadhikari7.com.np

Need for Learning

As with many other types of AI system, it is much more efficient to give the system enough knowledge to get it started, and then leave it to learn the rest for itself. We may even end up with a system that learns to be better than a human expert.

The ***general learning approach*** is to generate potential improvements, test them, and discard those which do not work. Naturally, there are many ways we might generate the potential improvements, and many ways we can test their usefulness. At one extreme, there are model driven (top-down) generators of potential improvements, guided by an understanding of how the problem domain works. At the other, there are data driven (bottom-up) generators, guided by patterns in some set of training data.

notes.arpanadhikari7.com.np

Machine Learning

Branch of AI that use algorithms to allow computer to evolve behaviours based on data collected from database or gathered through sensors

Focuses on prediction, based on known properties learned from the training data

The performance is usually evaluated with respect to the ability to reproduce known knowledge

notes.arpanadhikari7.com.np

Machine Learning

As regards machines, we might say, very broadly, that a machine learns whenever it changes its structure, program, or data (based on its inputs or in response to external information) in such a manner that its expected future performance improves.

Some of these changes, such as the addition of a record to a data base, fall comfortably within the province of other disciplines and are not necessarily better understood for being called learning. But, for example, when the performance of a speech-recognition machine improves after hearing several samples of a person's speech, we feel quite justified in that case saying that the machine has learned.

Machine learning usually refers to the changes in systems that perform tasks associated with artificial intelligence (AI).

Such tasks involve recognition, diagnosis, planning, robot control, prediction, etc.

The changes might be either enhancements to already performing systems or synthesis of new systems.

Available on notes.arpanadhikari7.com.np

Concept of Learning

Environment

- It refers to the nature and quality of the information given to the learning element
- The nature of information depends on its level
  - High level: information is abstract and deals with broad class of problems
  - Low level: information is detailed and deals with single problem
- Quality of information involves noise free, ordered and reliable information

notes.arpanadhikari7.com.np

Concept of Learning

Learning Element

- Acquire new knowledge through learning element
- Learning may be of any type discussed above

Problem Generator/ Knowledge Base

- Stores the information about the problems and solutions are suggested
- Knowledge Base should be
  - Expressive: Knowledge should be represented in easy and understandable way
  - Modifiable: Must be easy to update or add new data in the knowledge base
  - Extendibility: the knowledge base should have feature to change its structure that should be well defined

notes.arpanadhikari7.com.np

Concept of Learning

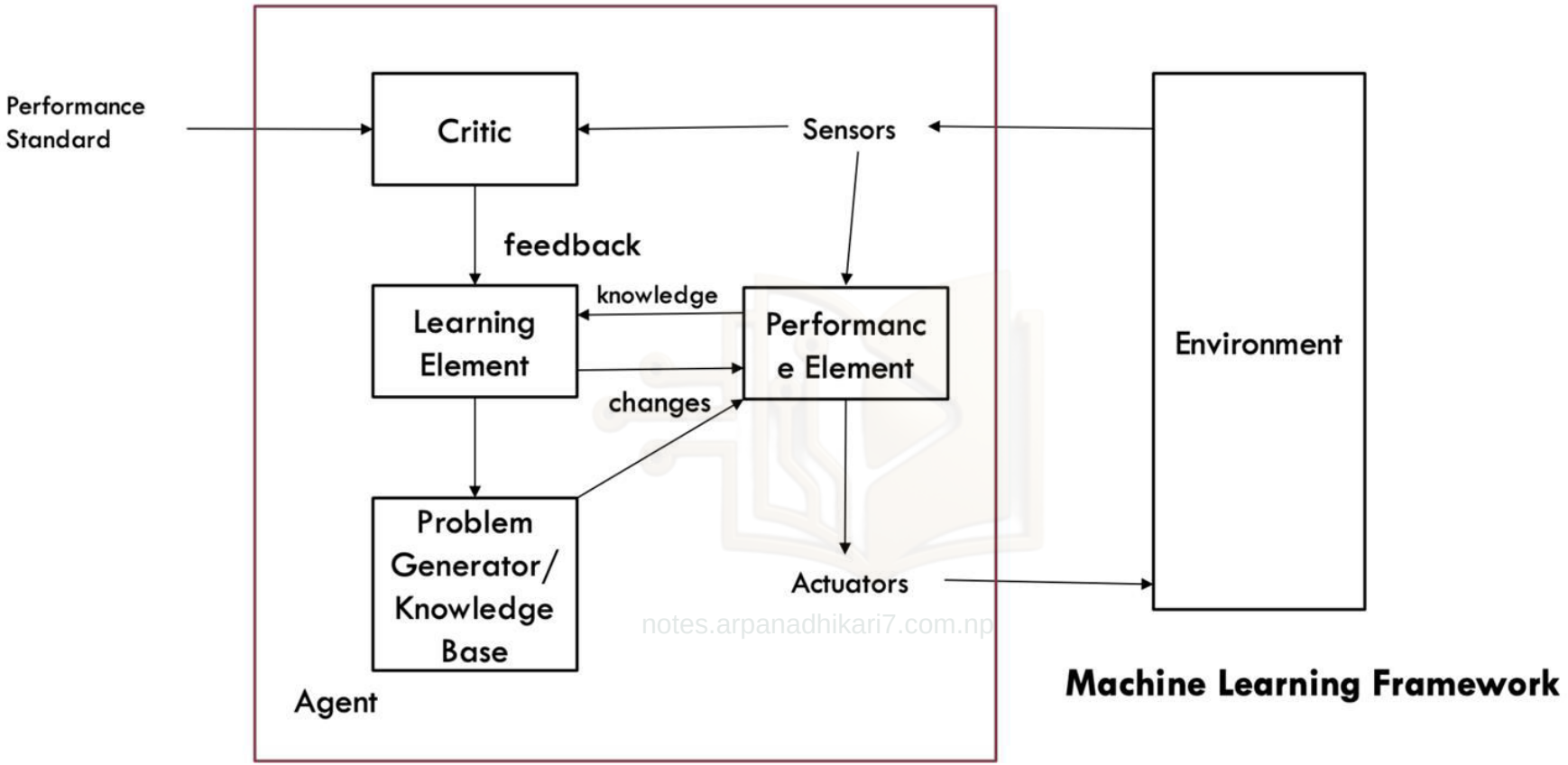
Performance Element

- This part analyses how complex the learning is and how learning is being performed
- Complexity depends on the type of task to be performed It must send feedback to the learning system as well so that evaluation of overall performance could be done
- The learning element should have access to all internal actions of the performance elements

notes.arpanadhikari7.com.np

Sensors and Actuators

- Sensors collect information from environment
- Actuators implement the suggested action



Machine Learning Framework

notes.arpanadhikari7.com.np

Rote Learning

Rote learning is a technique which focuses on memorization

It avoids understanding the inner complexities and inferences of the subject that is being learned and instead focuses on memorizing the material

The major practice involved in rote learning is repetition

notes.arpanadhikari7.com.np

Learning by explanation

Explanation-based learning (EBL) uses a domain theory to construct an explanation of the training example, usually a proof that the example logically follows from the theory

Using this proof the system filters the noise, selects only the relevant to the proof aspects of the domain theory, and organizes the training data into a systematic structure

This makes the system more efficient in later attempts to deal with the same or similar examples.

notes.arpanadhikari7.com.np

Basic Concept of EBL

- Target concept.
  - The task of the learning system is to find an effective definition of this concept.
  - Depending on the specific application the target concept could be a classification, theorem to be proven, a plan for achieving goal, or heuristic to make a problem solver more efficient.
- Training example.
  - This is an instance of the target concept
- Domain theory.
  - Usually this is a set of rules and facts representing domain knowledge.
  - They are used to explain how the training example is an instance of the target concept.
- Operationality criteria.
  - Some means to specify the form of the concept definition.

notes.arpanadhikari7.com.np

Learning by Analogy

Analogy is a powerful inference tool

Our language and reasoning are laden with analogy

Example: Last month share market was a roller coaster.

Bill is like a fire engine

So, AI must be able to grasp analogy for learning easily

notes.arpanadhikari7.com.np

It is used in different learning strategies like learning by advice taking, learning in problem solving, etc.

Learning by Analogy

Analogy is a powerful inference tool

Our language and reasoning are laden with analogy

Example: Last month share market was a roller coaster.

Bill is like a fire engine

So, AI must be able to grasp analogy for learning easily

notes.arpanadhikari7.com.np

It is used in different learning strategies like learning by advice taking, learning in problem solving, etc.

Learning by analogy

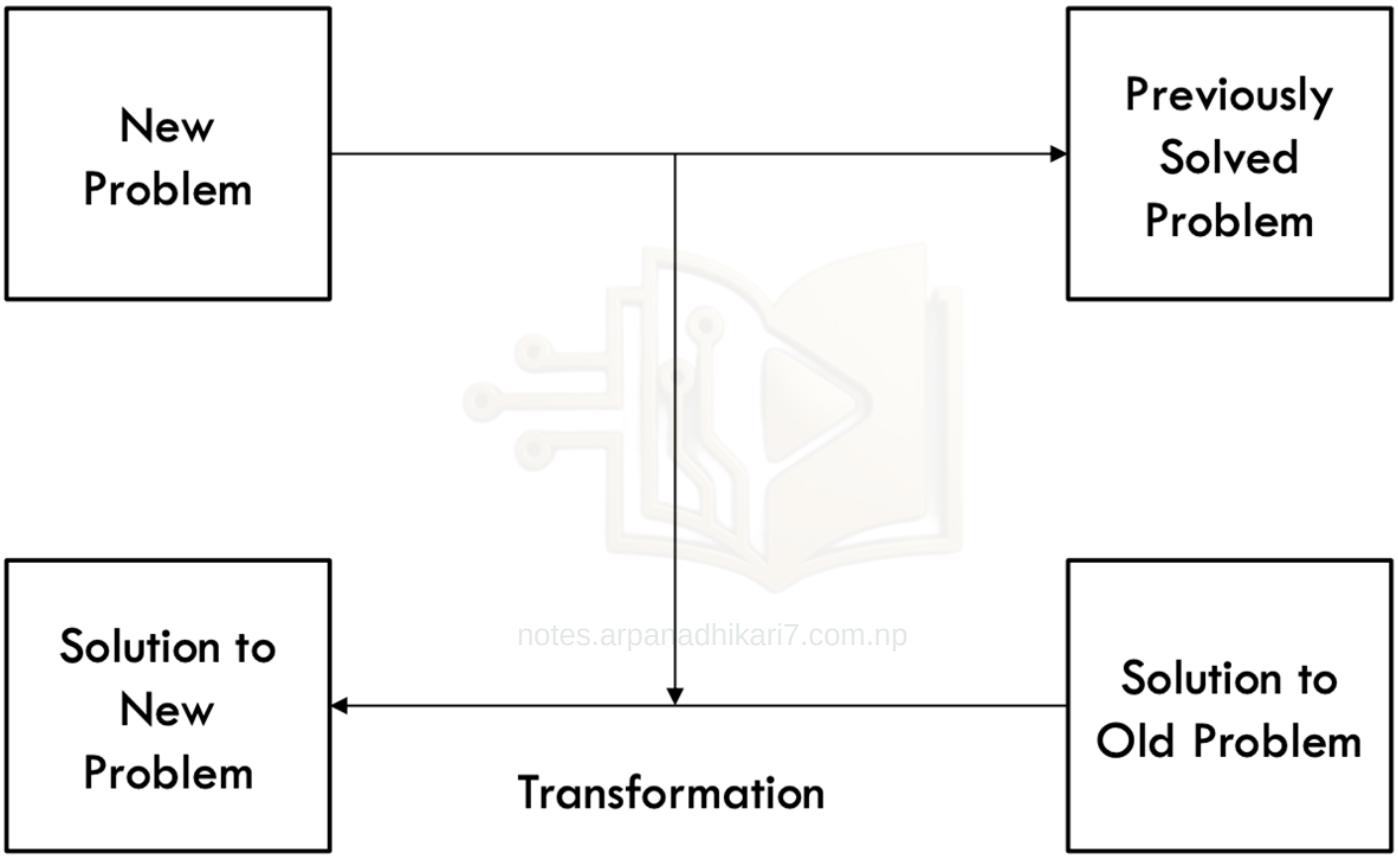
Two methods of Analogy Problem solving are:

Transformational Analogy : focuses on final solution

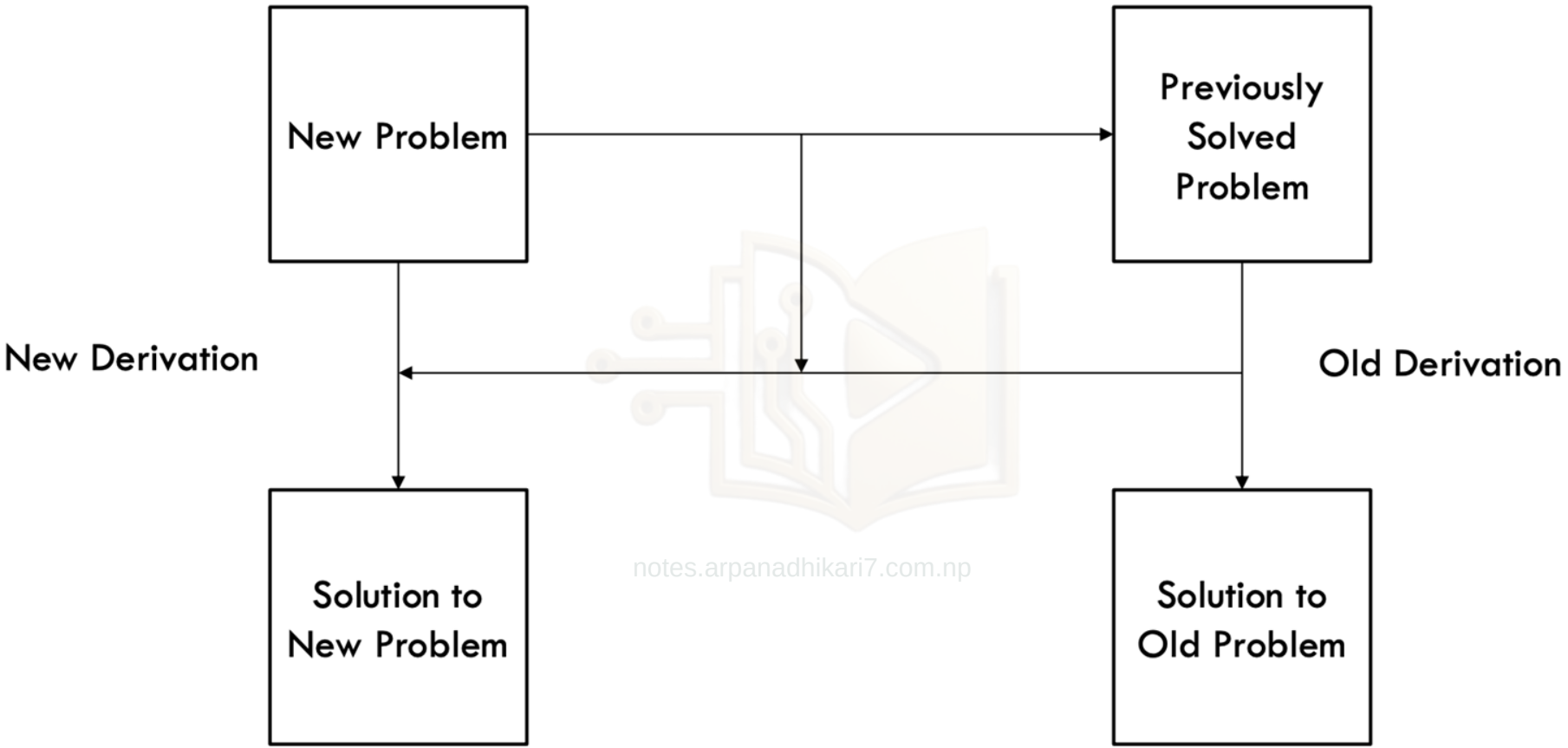
Derivational Analogy: focuses on process of problem solving

notes.arpanadhikari7.com.np

Transformational Analogy



notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Inductive learning

- Simplest form: learn a function from examples

- Idea:

Given:

- *f*: the *target function*
- Examples of *f* where an *example* is a pair $(x, f(x))$ (training data) and examples might have noise

Problem:

- find a *hypothesis h* such that $h \approx f$ (*h* is mostly consistent with *f*)

(This is a highly simplified model of real learning:

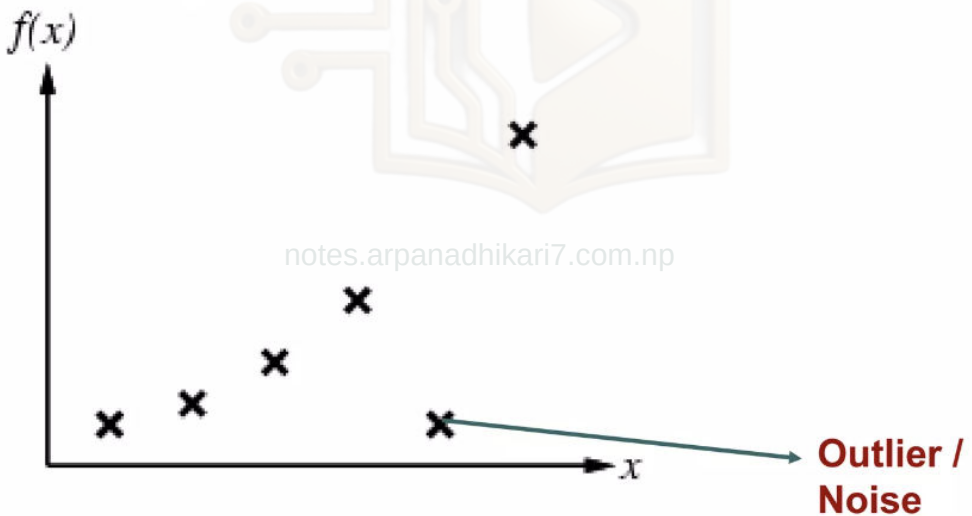
- Ignores prior knowledge
- Assumes that there are no missing examples)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Inductive learning method

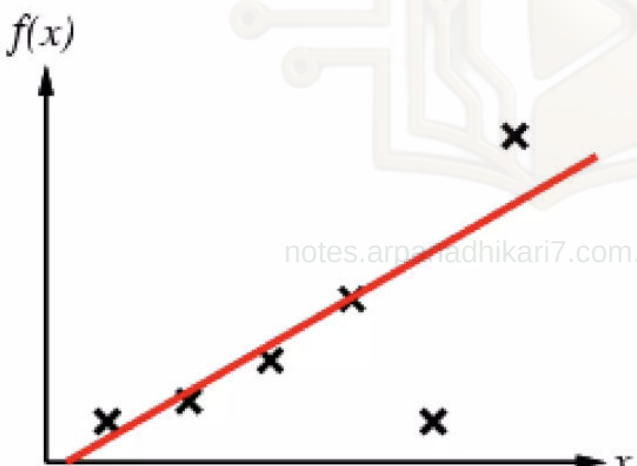
- Construct/adjust *h* to agree with *f* on training set
- (*h* is ***a consistent hypothesis*** if it agrees with *f* on all examples)
- E.g., curve fitting:



notes.arpanadhikari7.com.np

Inductive learning method (cont'd)

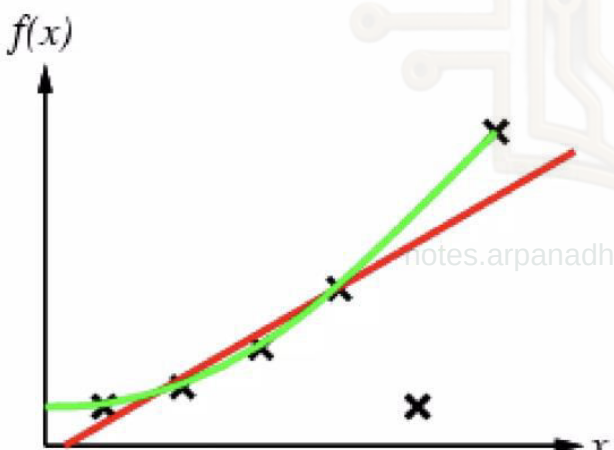
- Construct/adjust *h* to agree with *f* on training set
- (*h* is *consistent* if it agrees with *f* on all examples)
- E.g., curve fitting:



notes.arpanadhikari7.com.np

Inductive learning method (cont'd)

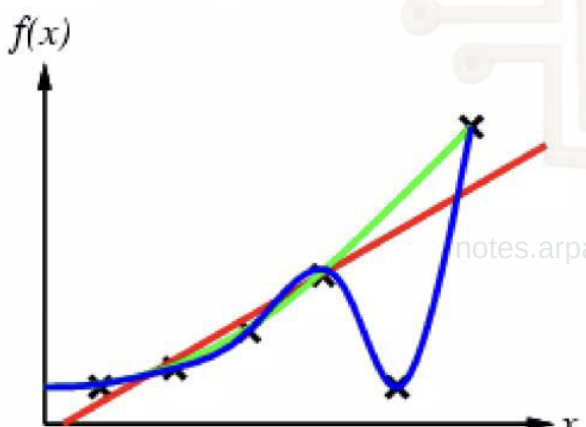
- Construct/adjust *h* to agree with *f* on training set
- (*h* is *consistent* if it agrees with *f* on all examples)
- E.g., curve fitting:



notes.arpanadhikari7.com.np

Inductive learning method (cont'd)

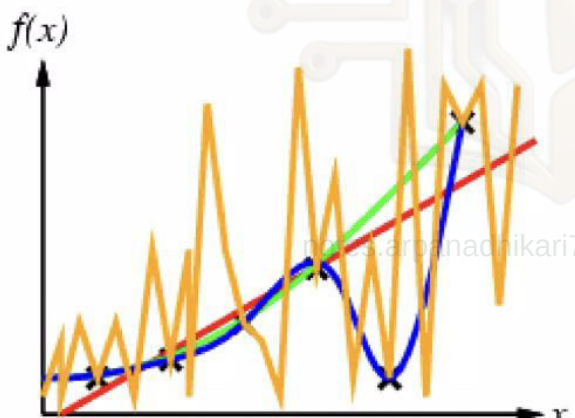
- Construct/adjust *h* to agree with *f* on training set
- (*h* is *consistent* if it agrees with *f* on all examples)
- E.g., curve fitting:



notes.arpanadhikari7.com.np

Inductive learning method (cont'd)

- Construct/adjust *h* to agree with *f* on training set
- (*h* is *consistent* if it agrees with *f* on all examples)
- E.g., curve fitting:



notes.arpanadhikari7.com.np

Supervised Learning

Supervised Learning is a type of machine learning where algorithms are trained on labeled data to make predictions or classify new data.

This approach involves using a dataset that includes both input features and their corresponding output labels.

The model learns from these labeled examples to identify patterns and relationships between inputs and outputs, enabling it to predict outputs for unseen data.

notes.arpanadhikari7.com.np

Classification problems

Predicting the category or class of an observation or data point is the goal of a classification issue in machine learning. The objective is to create a model that can learn to categorize fresh data based on correlations and patterns found in the training data.

The model is trained with labeled data, where the proper class or category of each observation is known, since classification is a supervised learning technique. The class of fresh, unknown data is then predicted using the model.

Classification issues might be binary—where there are only two classes that could exist—or multiclass—where there are multiple classes that could exist. Email spam categorization, image recognition, sentiment analysis, and disease diagnosis are a few typical instances of classification issues.

Available on notes.arpanadhikari7.com.np

Classification Problem

There are many algorithms that can be used for classification, including logistic regression, decision trees, support vector machines (SVMs), random forests, and neural networks. The choice of algorithm depends on factors such as the size and complexity of the data, the number of classes, and the accuracy and interpretability requirements of the model.

notes.arpanadhikari7.com.np

Linear Regression in Machine Learning

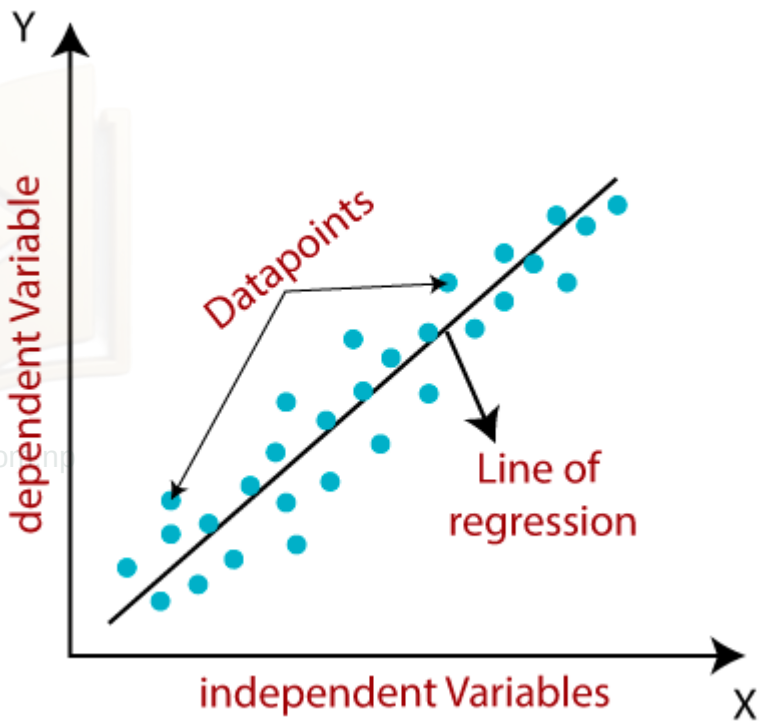
Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price,** etc.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (x) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

notes.arpanadhikari7.com.np

Linear Regression (cont.)

The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



notes.arpanadhikari7.com.np

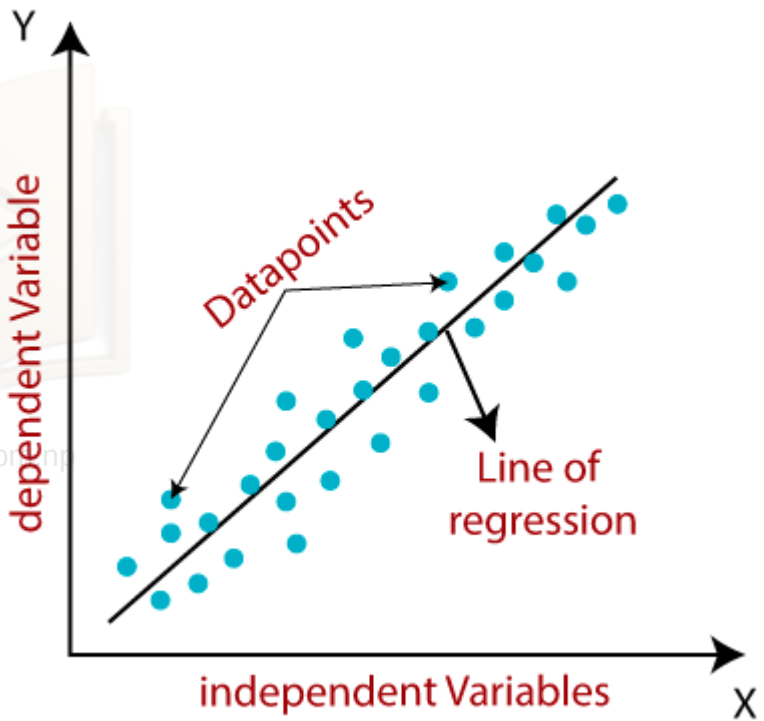
Linear Regression (cont.)

Mathematically, we can represent linear regression as:

$$y= a_0+a_1x+ \varepsilon$$

Y= Dependent Variable (Target Variable)
X= Independent Variable (predictor Variable)
a0= intercept of the line (Gives an additional degree of freedom)
a1 = Linear regression coefficient (scale factor to each input value).
ε = random error

The values for x and y variables are training datasets for Linear Regression model representation.



notes.arpanadhikari7.com.np

Types of Linear Regression

Linear regression can be further divided into two types of the algorithm:

- **Simple Linear Regression:**

  If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.

- **Multiple Linear regression:**

  If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

notes.arpanadhikari7.com.np

Finding the best fit-line

When working with linear regression, our main goal is to find the best fit line that means the error between predicted values and actual values should be minimized. The best fit line will have the least error.

The different values for weights or the coefficient of lines (a_0, a_1) gives a different line of regression, so we need to calculate the best values for a_0 and a_1 to find the best fit line, so to calculate this we use cost function.

Cost function-

- The different values for weights or coefficient of lines (a_0, a_1) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable. This mapping function is also known as **Hypothesis function**.

For Linear Regression, we use the **Mean Squared Error (MSE)** cost function, which is the average of squared error occurred between the predicted values and actual values. It can be written as:

$$MSE = 1\frac{1}{N}\sum_{i=1}^{n}(y_i - (a_1x_i + a_0))^2$$

notes.arpanadhikari7.com.np

Where,

N=Total number of observation
Yi = Actual value
$(a1x_i+a_0)$= Predicted value.

Problem Deninition:

Find a quadratic regression model for the following data:

X	Y
1	1
2	2
3	1.3
4	3.75
5	2.25

notes.arpanadhikari7.com.np

Solution:

Let the simple linear regression model be

$$y = a + bx$$

Steps to find **a** and **b,**

First, find the mean and covariance.

Means of x and y are given by,

$$\bar{x} = \frac{1}{n}\sum x_i$$

$$\bar{y} = \frac{1}{n}\sum y_i$$

notes.arpanadhikari7.com.np

The variance of x is given by,

$$\text{Var}\,(x) = \frac{1}{n-1}\sum(x_i - \bar{x_i})^2$$

The covariance of x and y, denoted by Cov(x, y)is defined as,

$$\text{Cov}(x, y) = \frac{1}{n-1}\sum(x_i - \bar{x})(y_i - \bar{y})$$

Now the values of a and b can be computed using the following formulas:

notes.arpanadhikari7.com.np

$$b = \frac{\text{Cov}\,(x, y)}{\text{Var}\,(x)}$$

$$a = \bar{y} - b\bar{x}$$

First, find the mean of x and y,

$$
\begin{aligned}
n &= 5 \\
\bar{x} &= \frac{1}{5}(1.0 + 2.0 + 3.0 + 4.0 + 5.0) \\
&= 3.0 \\
\bar{y} &= \frac{1}{5}(1.00 + 2.00 + 1.30 + 3.75 + 2.25) \\
&= 2.06
\end{aligned}
$$

Next, find the Covariance between x and y,

notes.arpanadhikari7.com.np

$$\mathrm{Cov}(x, y) = \frac{1}{n-1}\sum(x_i - \bar{x})(y_i - \bar{y})$$

$$
\begin{aligned}
\mathrm{Cov}\,(x, y) &= \frac{1}{4}[(1.0 - 3.0)(1.00 - 2.06) + \cdots + (5.0 - 3.0)(2.25 - 2.06)] \\
&= 1.0625
\end{aligned}
$$

Now find the variance of x,

$$\text{Var}(x) = \frac{1}{n-1}\sum(x_i - \bar{x}_i)^2$$

$$\text{Var}(x) = \frac{1}{4}[(1.0-3.0)^2 + \cdots + (5.0-3.0)^2]$$
$$= 2.5$$

Now, find the intercept and coefficients,

$$b = \frac{\text{Cov}(x,y)}{\text{Var}(x)}$$

$$a = \bar{y} - b\bar{x}$$

$$b = \frac{1.0625}{2.5}$$
$$= 0.425$$

$$a = 2.06 - 0.425 \times 3.0$$
$$= 0.785$$

notes.arpanadhikari7.com.np

Therefore, the linear regression model for the data is,
y = 0.785 + 0.425 x

Multiple Linear Regression

Multiple linear regression is used to estimate the relationship between two or more independent variables and one dependent variable. You can use multiple linear regression when you want to know:

1. How strong the relationship is between two or more independent variables and one dependent variable (e.g. how rainfall, temperature, and amount of fertilizer added affect crop growth).
2. The value of the dependent variable at a certain value of the independent variables (e.g. the expected yield of a crop at certain levels of rainfall, temperature, and fertilizer addition).

notes.arpanadhikari7.com.np

Multiple linear regression formula

The formula for a multiple linear regression is:

$$y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n + \epsilon$$

- y = the predicted value of the dependent variable
- B_0 = the y-intercept (value of y when all other parameters are set to 0)
- $B_1 X_1$ = the regression coefficient (B_1) of the first independent variable (X_1) (a.k.a. the effect that increasing the value of the independent variable has on the predicted y value)
- ... = do the same for however many independent variables you are testing
- $B_n X_n$ = the regression coefficient of the last independent variable
- ϵ = model error (a.k.a. how much variation there is in our estimate of y)

- The multiple regression of two variables x1 and x2 is given as follows:

$$y = f(x_1, x_2)$$

$$y = a_0 + a_1 x_1 + a_2 x_2$$

- In general, this is given for 'n' independent variables as:

$$y = f(x_1, x_2, \ldots . x_n)$$

$$y = a_0 + a_1 x_1 + a_2 x_2 + \ldots \ldots + a_n x_n + \varepsilon$$

- Here, $x_1, x_2, \ldots . x_n$ are predictor variables, y is the dependent variable, $(a_0, a_1, a_2, \ldots . a_n)$ are the coefficients of the regression equation and ε is the error term.

Available on notes.arpanadhikari7.com.np

Instance Based Learning

Instance-based learning (IBL) is a type of machine learning where the model is not explicitly trained on a set of examples, but instead stores the examples themselves (called instances) and uses them to make predictions for new, unseen data.

In IBL, when a new data point is presented to the model for prediction, it searches the stored instances for the ones that are most similar to the new data point. These similar instances are used to make a prediction for the new data point. The similarity between instances is typically measured using a distance metric, such as Euclidean distance or cosine similarity.

Available on notes.arpanadhikari7.com.np

The most common instance-based learning algorithm is the k-nearest neighbors (KNN) algorithm. KNN works by storing all the training instances and their associated labels.

When presented with a new instance, the algorithm finds the k closest instances in the training set (i.e., the k nearest neighbors) and takes a majority vote of their associated labels to make a prediction for the new instance.

notes.arpanadhikari7.com.np

K-Nearest Neighbor

- K-nearest neighbors (kNN) is a supervised machine learning algorithm that can be used for classification as well as regression.
- kNN in a *real life example*.
- People tend to be affected by the people around them. Our behaviour is guided by the friends we grew up with. So, If you grow up with people who love sports, it is highly likely that you will end up loving sports.
- It classifies the data points based on the similarity measure (e.g. distance measures, mostly Euclidean distance).

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

KNN

- K- NN algorithm is based on the principle that, “the similar things exist closer to each other or Like things are near to each other.”
- It classifies new cases based on a similarity measure (i.e. mostly Euclidean distance)

Distance functions

Euclidean $\sqrt{\sum_{i=1}^{k}(x_i - y_i)^2}$

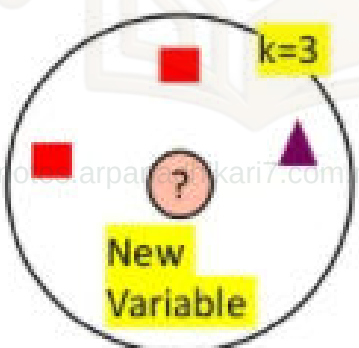
Manhattan $\sum_{i=1}^{k}|x_i - y_i|$

Minkowski $\left(\sum_{i=1}^{k}(|x_i - y_i|)^q\right)^{1/q}$

Available on notes.arpanadhikari7.com.np

What is K?

- ‘K’ refers to the number of neighbors to consider for classification
- Or, K is a parameter in KNN that refers to the number of nearest neighbors to include in the majority voting process.



Available on notes.arpanadhikari7.com.np

How to choose k

- Choosing the right value of k is a process called parameter tuning and is important for better accuracy.
- If the value of ‘K’ is small then it causes Low Bias, High variance i.e. overfitting of the model.
- In the same way, if ‘K’ is very large then it leads to High Bias, Low variance i.e. underfitting of the model.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Algorithm for KNN

1. Load the given data file into your program
2. Initialize the number of neighbors to be considered i.e. 'K' (must be odd).
3. Now for each tuple (entries or data point) in the data file we perform:
    1. Calculate the distance between the data point (tuple) to be classified and each data points in the given data file.
    2. Then add the distances corresponding to data points (data entries) in the given data file (probably by adding a column for distance).
    3. Sort the data in the data file from smallest to largest (in ascending order) by the distances.
4. Pick the first K entries from the sorted collection of data.
5. Observe the labels of the selected K entries.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

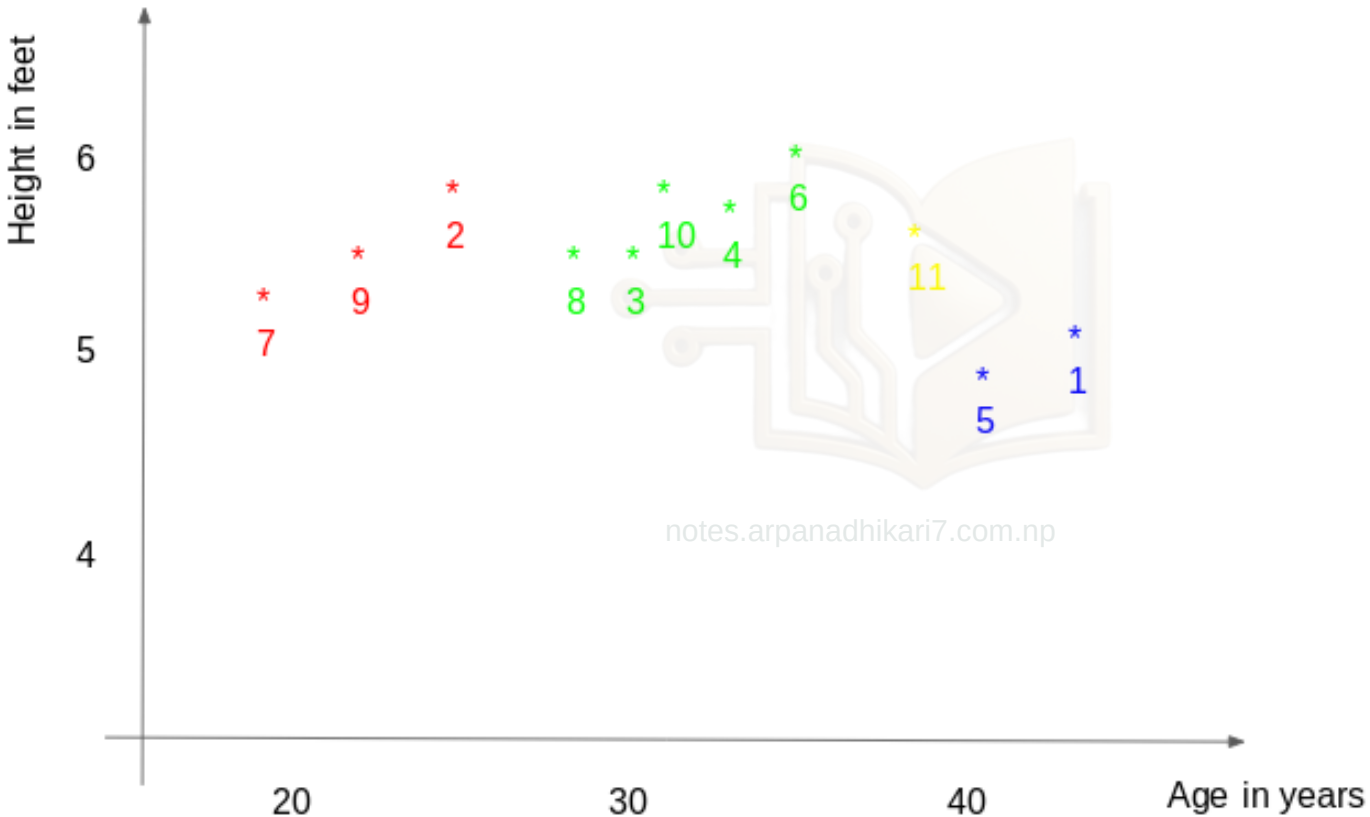
Let's observe the following dataset

ID	Height	Age	Weight
1	5	45	77
2	5.11	26	47
3	5.6	30	55
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
7	5.3	19	40
8	5.8	28	60
9	5.5	23	45
10	5.6	32	58
11	5.5	38	?

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

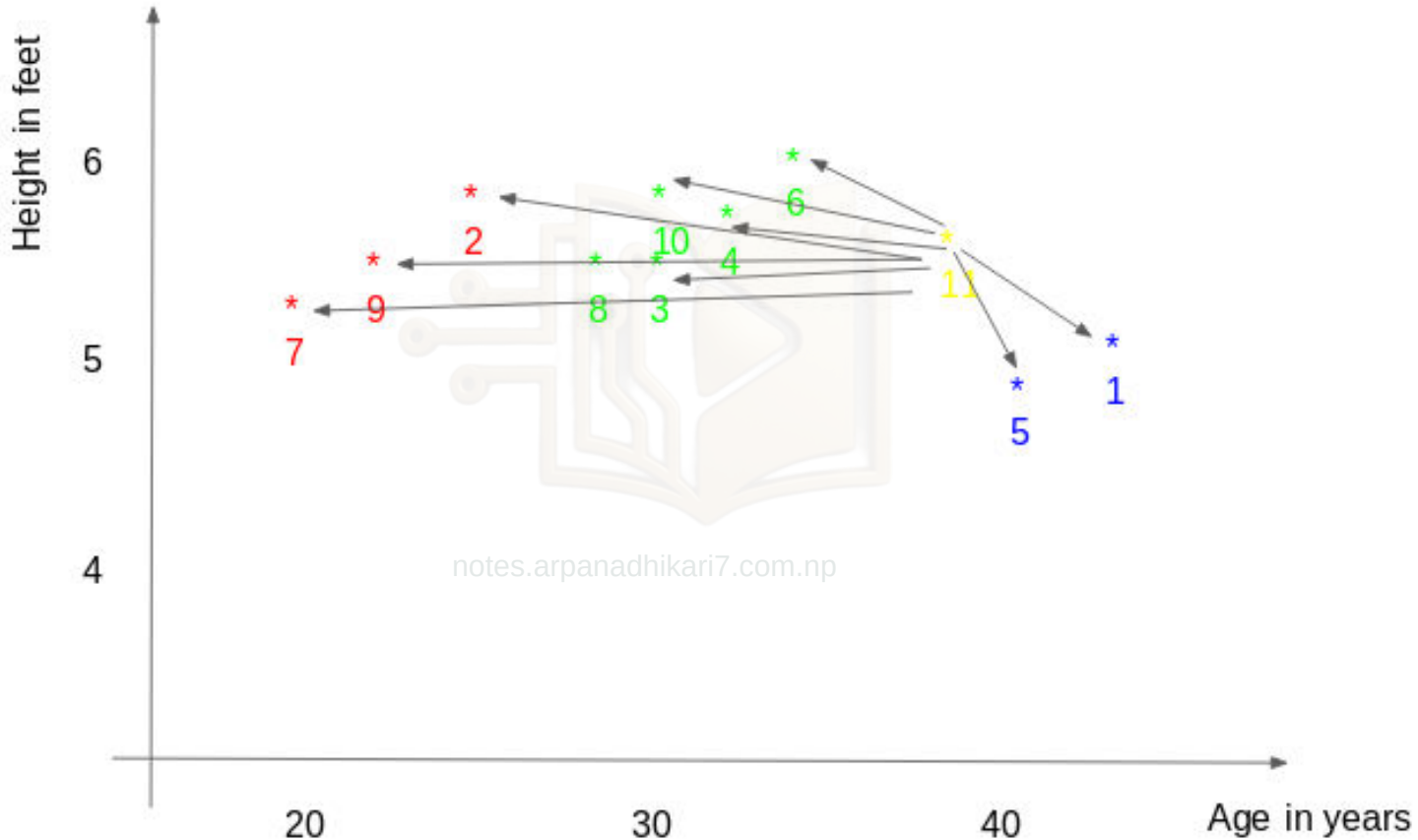
Initial Datasets observed (height vs age)



notes.arpanadhikari7.com.np

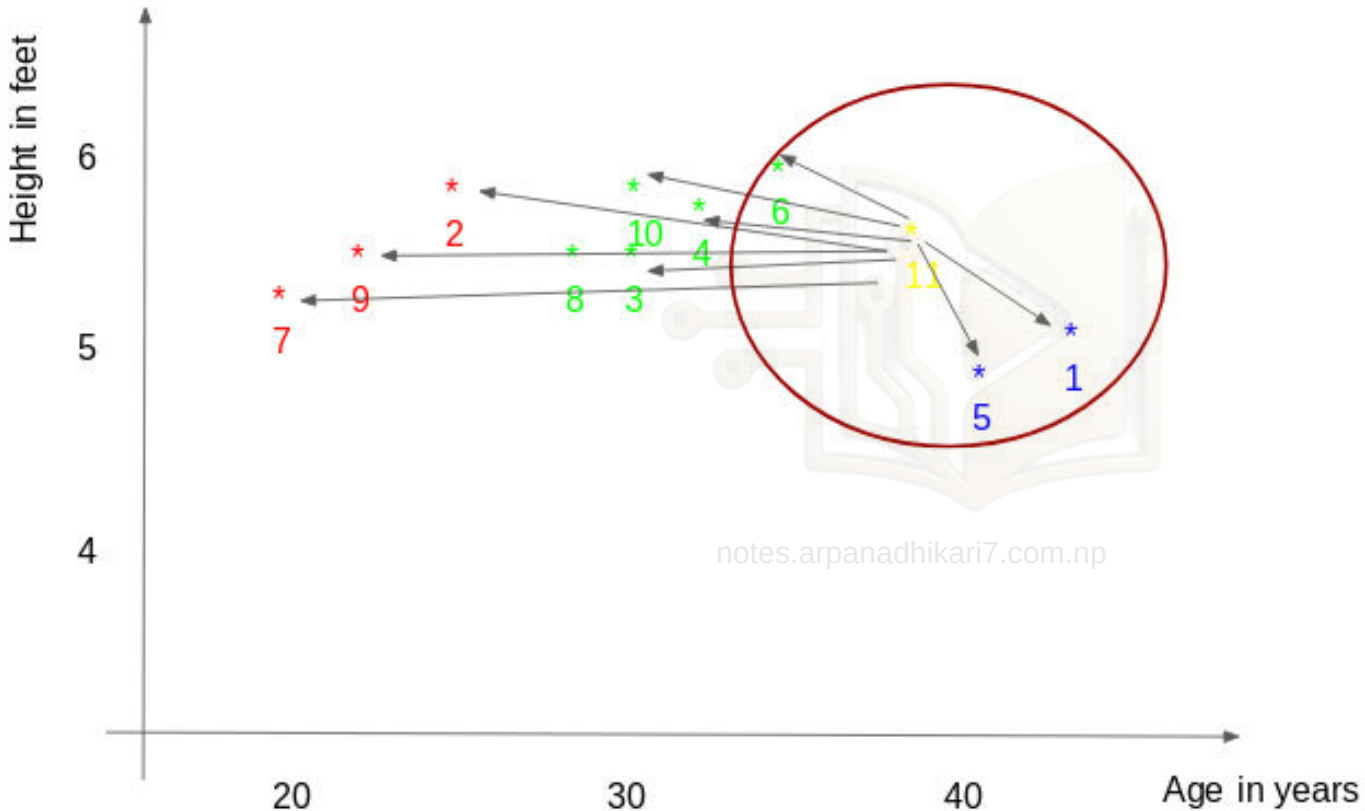
Available on notes.arpanadhikari7.com.np

Observe the distance between new point and training sets



Available on notes.arpanadhikari7.com.np

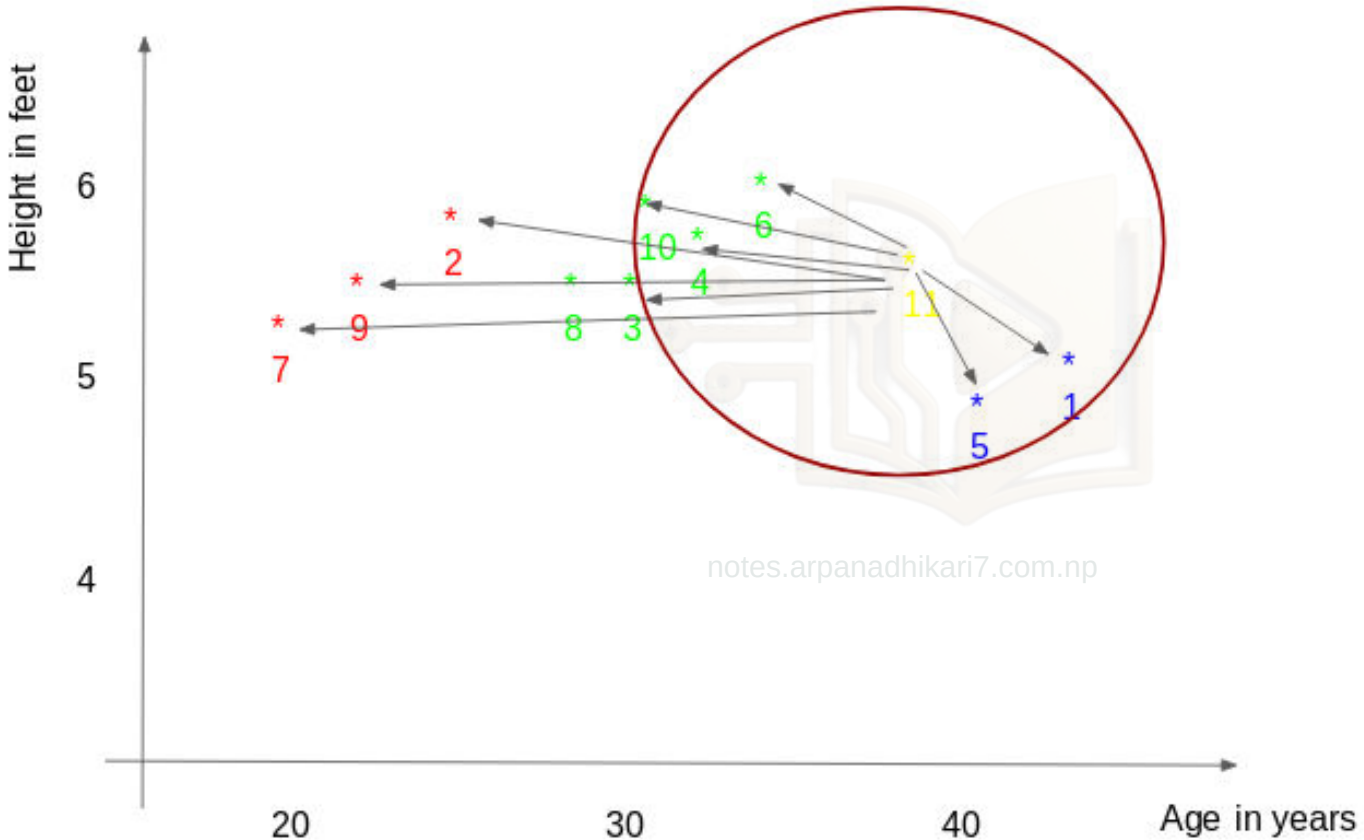
If value of K =3 ,then it reaches out for the nearest three training sets



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

If value of K =5 ,then it reaches out for the nearest five training sets



notes.arpanadhikari7.com.np

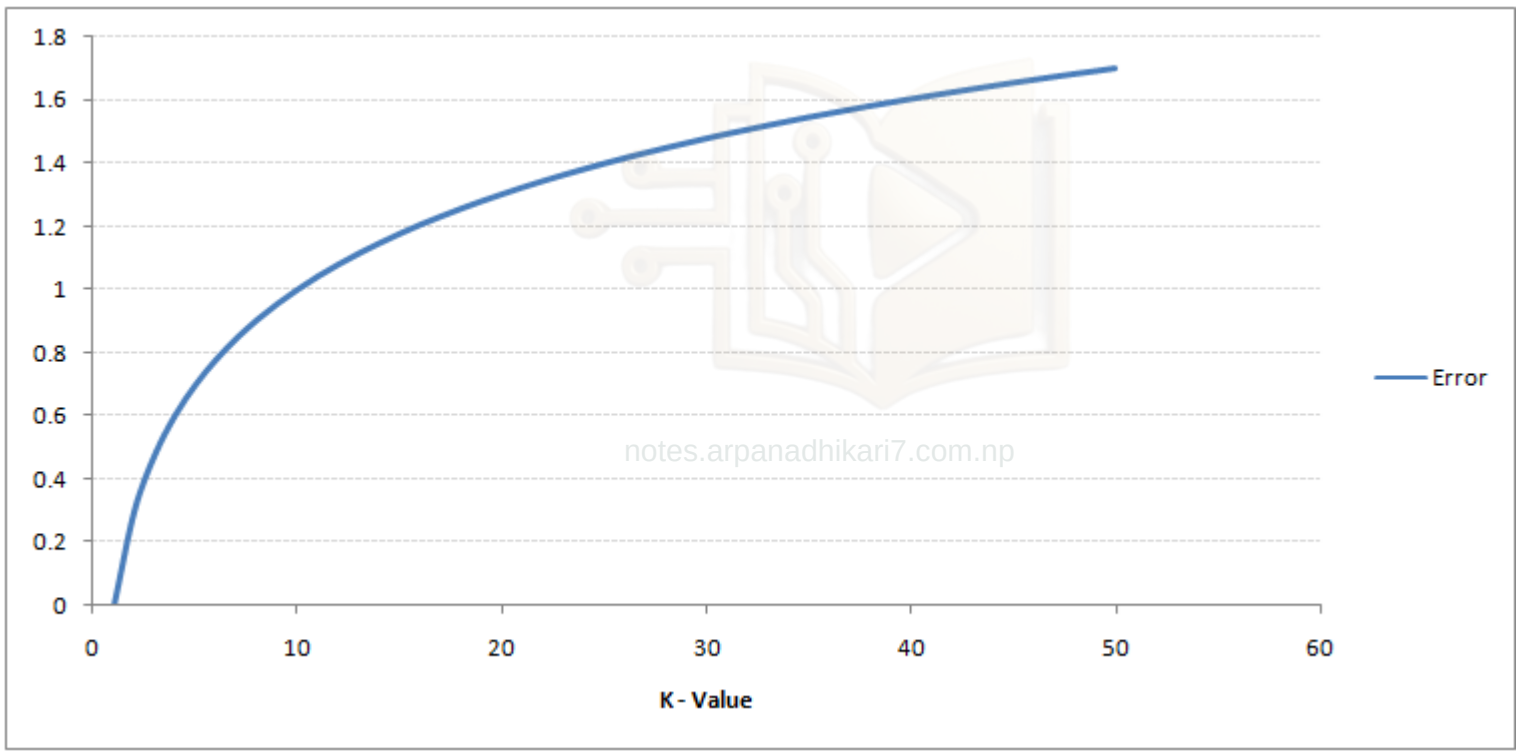
Available on notes.arpanadhikari7.com.np

- From the above observation, we can see that for different values of K, we can get different results for new data point.
- So how to determine the optimum value of k?
- By error calculation of training and validation set, we can find the optimum value of k.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

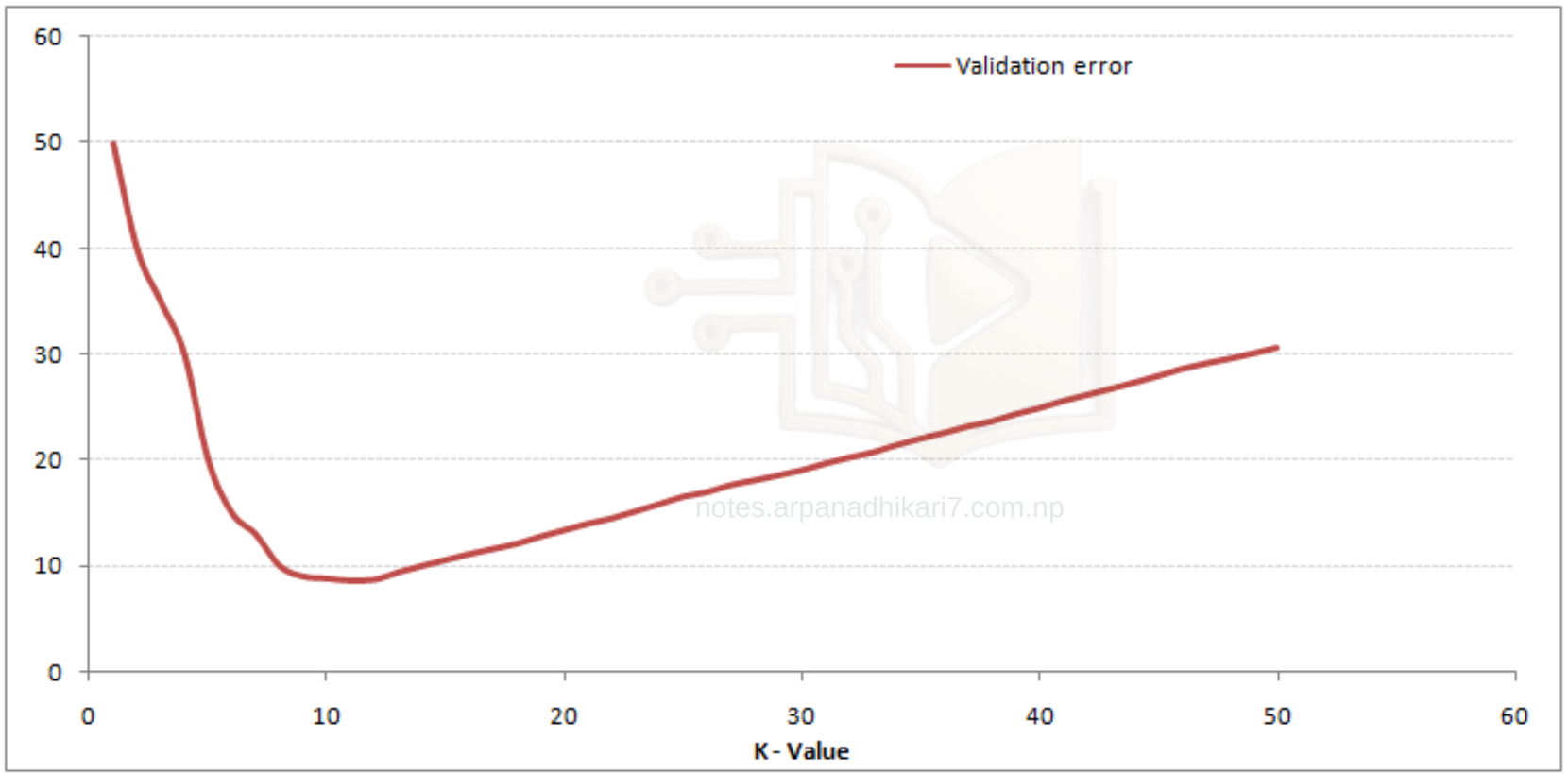
Error graph on training set



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Error graph on validation set



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- For a very low value of k (suppose k=1), the model overfits on the training data, which leads to a high error rate on the validation set.
- On the other hand, for a high value of k, the model performs poorly on both train and validation set.
- If you observe closely, the validation error curve reaches a minima at a value of k = 9.
- This value of k is the optimum value of the model (it will vary for different datasets).
- This curve is known as an ‘**elbow curve**‘ (because it has a shape like an elbow) and is usually used to determine the k value.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Pros of KNN algorithm

It is very simple algorithm to understand and interpret.

It is very useful for nonlinear data because there is no assumption about data in this algorithm.

It is a versatile algorithm as we can use it for classification as well as regression.

It has relatively high accuracy but there are much better supervised learning models than KNN.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cons of KNN Algorithm

It is computationally a bit expensive algorithm because it stores all the training data.

High memory storage required as compared to other supervised learning algorithms.

Prediction is slow in case of big N.

It is very sensitive to the scale of data as well as irrelevant features.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Applications of KNN

Banking System

KNN can be used in banking system to predict weather an individual is fit for loan approval? Does that individual have the characteristics similar to the defaulters one?

Calculating Credit Ratings

KNN algorithms can be used to find an individual's credit rating by comparing with the persons having similar traits.

Politics

With the help of KNN algorithms, we can classify a potential voter into various classes like "Will Vote", "Will not Vote", "Will Vote to Party 'Congress', "Will Vote to Party 'UML'.

Other areas in which KNN algorithm can be used are Speech Recognition, Handwriting Detection, Image Recognition and Video Recognition.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Apply K nearest neighbor classifier to predict the diabetic patient with the given features BMI, Age. If the training examples are,

BMI	Age	Sugar
33.6	50	1
26.6	30	O
23.4	40	O
43.1	67	O
35.3	23	1
35.9	67	1
36.7	45	1
25.7	46	O
23.3	29	O
31	56	1

notes.arpanadhikari7.com.np

Assume K=3,

Test Example BMI=43.6, Age=40, Sugar=?

The given training dataset has 10 instances with two features BMI (Body Mass Index) and Age. Sugar is the target label.

The target label has two possibilities 0 and 1. 0 means the diabetic patient has no sugar and 1 means the diabetic patient has sugar.

Given the dataset and new test instance, we need to find the distance from the new test instance to every training example.

Here we use the euclidean distance formula to find the distance.

$$Distance = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

notes.arpanadhikari7.com.np

BMI	Age	Sugar	Formula	Distance
33.6	50	1	√((43.6-33.6)^2+(40-50)^2)	14.14
26.6	30	O	√((43.6-26.6)^2+(40-30)^2)	19.72
23.4	40	O	√((43.6-23.4)^2+(40-40)^2)	20.20
43.1	67	O	√((43.6-43.1)^2+(40-67)^2)	27.00
35.3	23	1	√((43.6-35.3)^2+(40-23)^2)	18.92
35.9	67	1	√((43.6-35.9)^2+(40-67)^2)	28.08
36.7	45	1	√((43.6-36.7)^2+(40-45)^2)	8.52
25.7	46	O	√((43.6-25.7)^2+(40-46)^2)	18.88
23.3	29	O	√((43.6-23.3)^2+(40-29)^2)	23.09
31	56	1	√((43.6-31)^2+(40-56)^2)	20.37

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Once you calculate the distance, the next step is to find the nearest neighbors based on the value of k. In this case, the value of k is 3. Hence we need to find 3 nearest neighbors.

BMI	Age	Sugar	Distance	Rank
33.6	50	1	14.14	2
26.6	30	O	19.72	
23.4	40	O	20.20	
43.1	67	O	27.00	
35.3	23	1	18.92	
35.9	67	1	28.08	
36.7	45	1	8.52	1
25.7	46	O	18.88	3
23.3	29	O	23.09	
31	56	1	20.37	

notes.arpanadhikari7.com.np

Now, we need to apply the majority voting technique to decide the resulting label for the new example.

Here the 1st and 2nd nearest neighbors have target label 1 and the 3rd nearest neighbor has target label 0.
Target label 1 has the majority.
Hence the new example is classified as 1, That is the diabetic patient has **Sugar**.

Available on notes.arpanadhikari7.com.np

Unsupervised Learning

Unsupervised Learning is a type of machine learning where algorithms are used to identify patterns, groupings, or relationships in data without prior knowledge of the expected output.

Unlike supervised learning, which relies on labeled data to train models, unsupervised learning operates on unlabeled data, making it ideal for exploratory data analysis, customer segmentation, and image recognition.

notes.arpanadhikari7.com.np

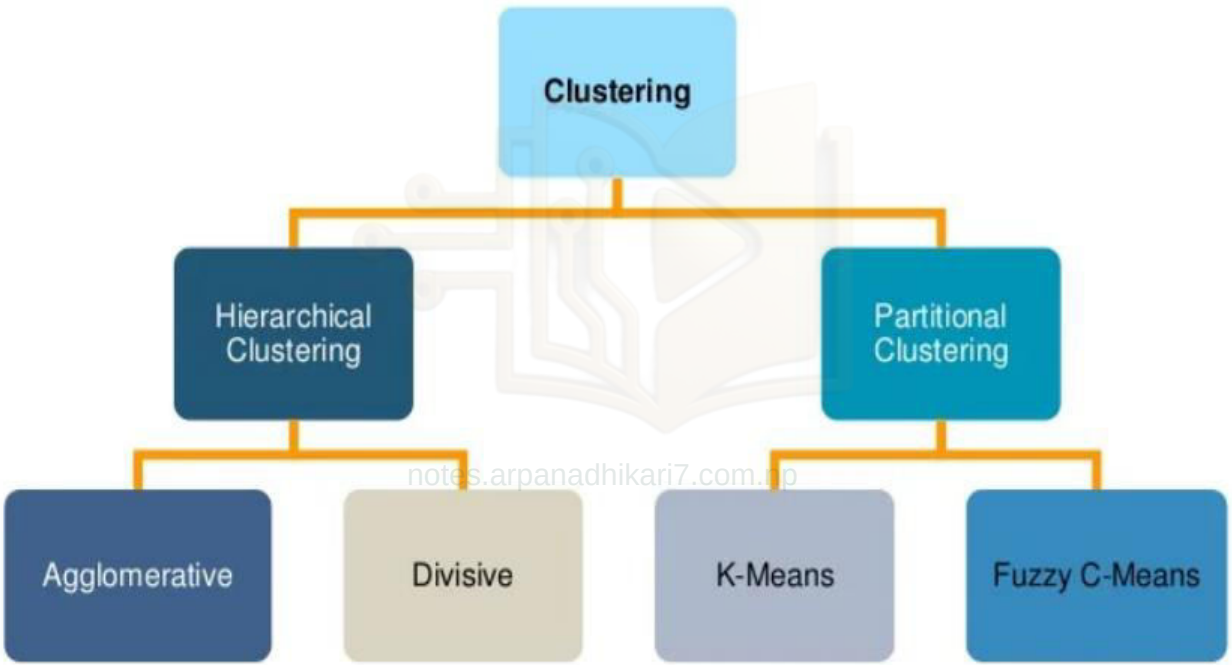
Introduction – What is Clustering?

- Clustering is the classification of objects into different groups, or more precisely, the partitioning of a data set into subsets (clusters), so that the data in each subset(ideally) share some common trait-often according to some defined distance measure.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types of Clustering



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

K Means Clustering

- The k-means algorithm is an algorithm to cluster n objects based on attributes into k partitions, where k<n.
- K-means clustering is an unsupervised machine learning algorithm for clustering ‘n’ observations into ‘k’ clusters where k is predefined or user-defined constant.
- The main idea is to define k centroids, one for each cluster.
- It is similar to the expectation- maximization algorithm for mixtures of Gaussians in that they both attempt to find the centers of natural clusters in the data.

Available on notes.arpanadhikari7.com.np

- It assumes that the object attributes form a vector.
- An algorithm for partitioning (or clustering) N data points into k disjoint subsets S_j containing data points so as to minimize the sum of squares criterion

$$J = \sum_{j=1}^{K} \sum_{n \in S_j} |x_n - \mu_j|^2,$$

where x_n is a vector representing the nth data point and u_j is the geometric centroid of the data points in S_j.

- **It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.**

Available on notes.arpanadhikari7.com.np

- It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.

- The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.

Available on notes.arpanadhikari7.com.np

The k-means clustering algorithm mainly performs two tasks:

- **Determines the best value for K center points or centroids by an iterative process.**

- **Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.**

notes.arpanadhikari7.com.np

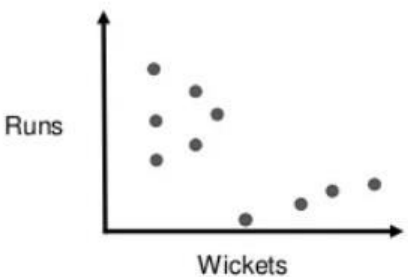
Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

Available on notes.arpanadhikari7.com.np

Assign data points

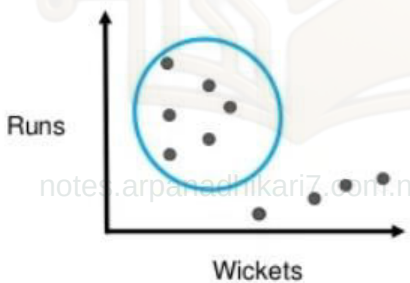
Here, we have our dataset with x and y coordinates

Now, we want to cluster this data using **K-Means**



Cluster 1

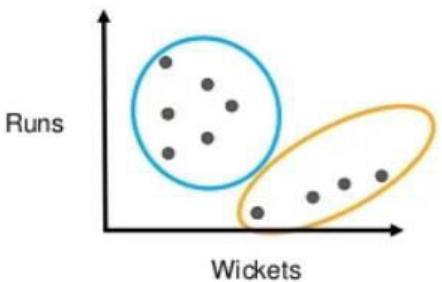
We can see that this cluster has players with high runs and low wickets



notes.arpanadhikari7.com.np

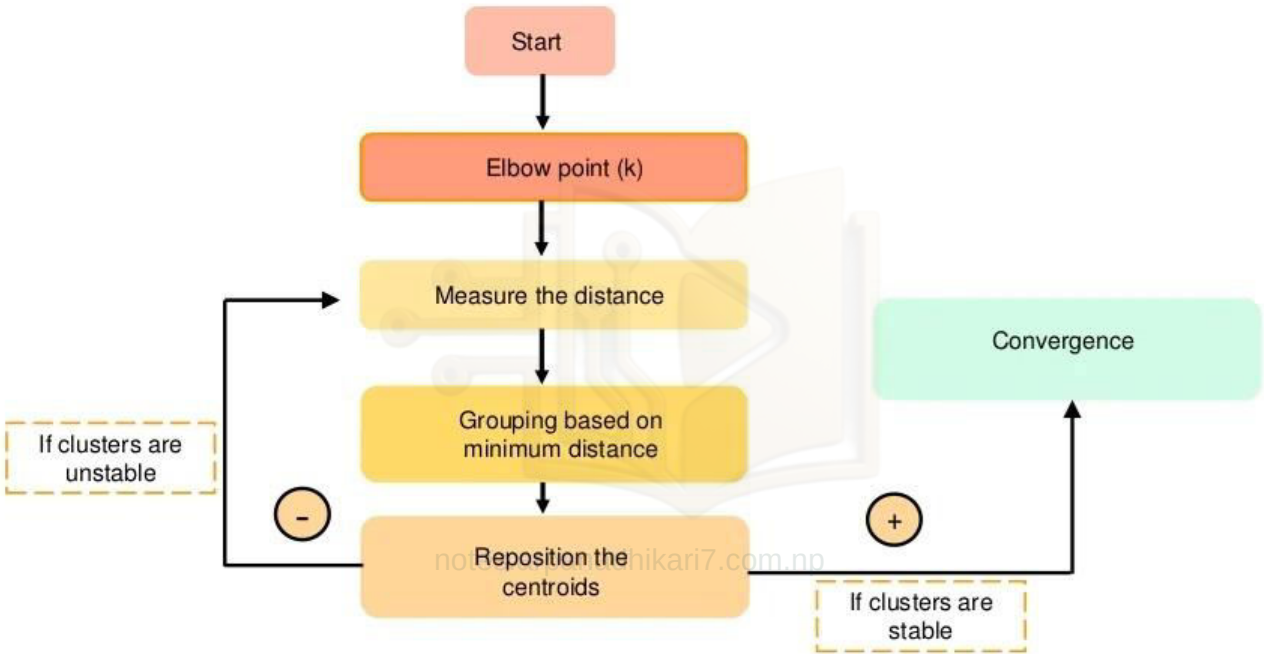
Cluster 2

And here, we can see that this cluster has players with high wickets and low wickets



Available on notes.arpanadhikari7.com.np

How does k-means clustering works?



Available on notes.arpanadhikari7.com.np

- Elbow point
- Measure the distance
- Grouping
- Reposition the centroids
- Convergence

- Let's say, you have a dataset for a **Grocery shop**



- Now, the important question is, "***how would you choose the optimum number of clusters?"***



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Elbow point

Measure the distance

Grouping

Reposition the centroids

Convergence

- The best way to do this is by **Elbow method**
- The idea of the elbow method is to run K-Means clustering on the dataset where 'k' is referred as number of clusters
- Within sum of squares (WSS) is defined as the sum of the squared distance between each member of the cluster and its centroid



$$WSS = \sum_{i=1}^{m} (x_i - c_i)^2$$

notes.arpanadhikari7.com.np

Where x_i = data point and c_i = closest point to centroid

Available on notes.arpanadhikari7.com.np

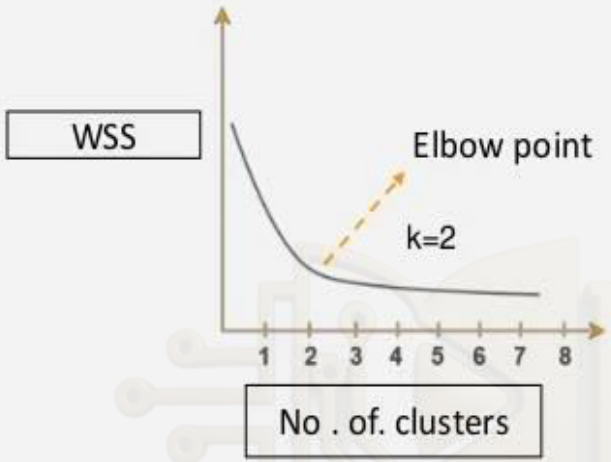
Elbow point

Measure the distance

Grouping

Reposition the centroids

Convergence



- Now, we draw a curve between ***WSS*** (within sum of squares) and the ***number of clusters***
- Here, we can see a very slow change in the value of WSS after k=2, so you should take that elbow point value as the final number of clusters

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Elbow point

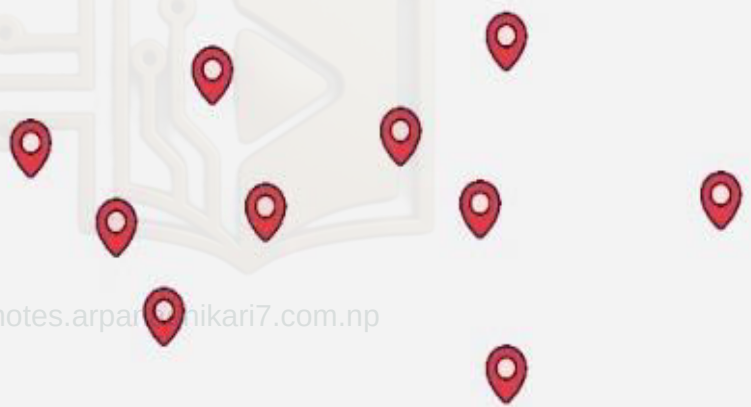
Measure the distance

Grouping

Reposition the centroids

Convergence

Step 1: The given data points below are assumed as **delivery points**



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Elbow point

Measure the distance

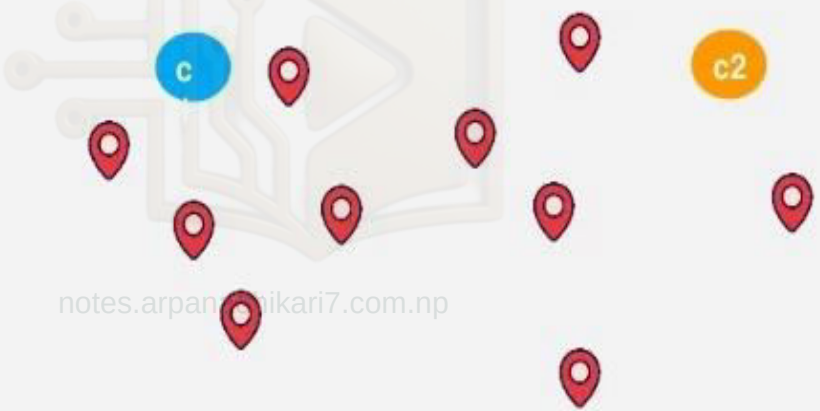
Grouping

Reposition the centroids

Convergence

Step 2: We can randomly initialize two points called the cluster centroids,

Euclidean distance is a distance measure used to find out which data point is closest to our centroids



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Elbow point

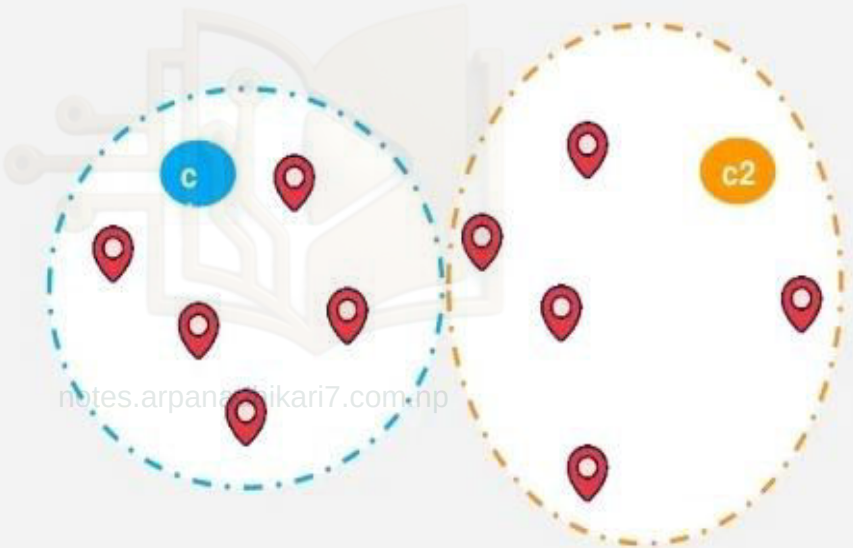
Measure the distance

Grouping

Reposition the centroids

Convergence

Step 3: Based upon the distance from c1 and c2 centroids, the data points will group itself into clusters



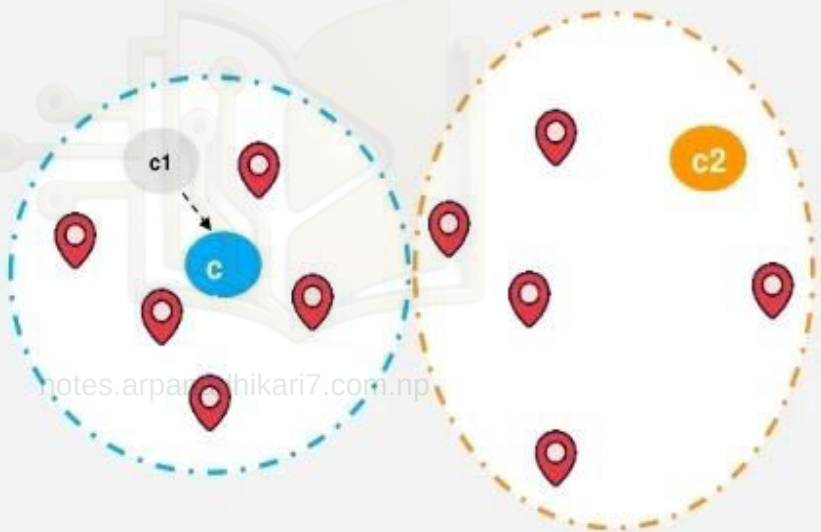
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Elbow point
- Measure the distance
- Grouping
- Reposition the centroids
- Convergence

Step 4: Compute the centroid of data points inside blue cluster

Step 5: Reposition the centroid of the blue cluster to the new centroid



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Elbow point

Measure the distance

Grouping

Reposition the centroids

Convergence

Step 6: Now, compute the centroid of data points inside orange cluster

Step 7: Reposition the centroid of the orange cluster to the new centroid



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Elbow point

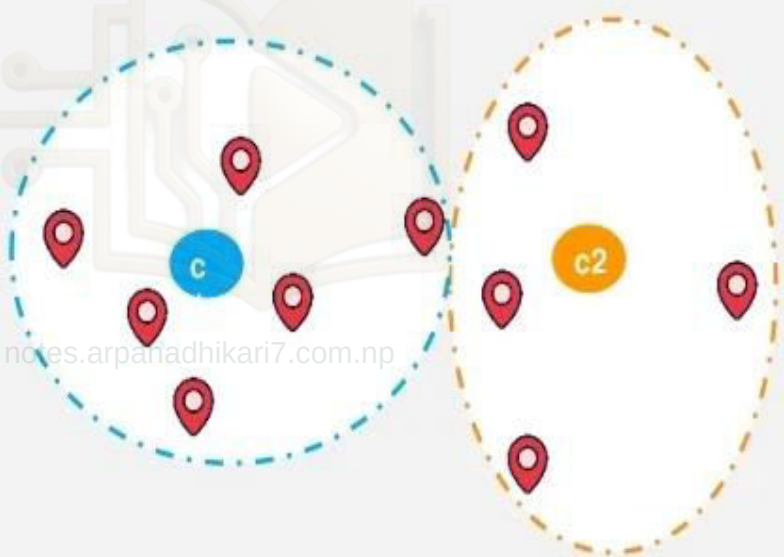
Measure the distance

Grouping

Reposition the centroids

Convergence

Step 8: Once the clusters become static, K-Means clustering algorithm is said to be converged



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

K-means clustering algorithm:

Assuming we have inputs x1,x2,x3,…, and value of **K,**

Step 1 : Pick K random points as cluster centers called centroids

Step 2 : Assign each xi to nearest cluster by calculating its distance to each centroid

Step 3 : Find new cluster center by taking the average of the assigned points

Step 4 : Repeat Step 2 and 3 until none of the cluster assignments change

Available on notes.arpanadhikari7.com.np

Step 1 :

We randomly pick **K** cluster centers (centroids). Let's assume these are c1,c2,…,ckc1,c2,…,ck, and we can say that;

$$C = c_1, c_2, \ldots c_k$$

C is the set of all centroids.

Step 2:

In this step, we assign each data point to closest center, this is done by calculating Euclidean distance

$$\arg\min_{c_i \in C} dist(\quad, x_i)^2$$

notes.arpanadhikari7.com.np

Where **dist()** is the Euclidean distance.

Available on notes.arpanadhikari7.com.np

Step 3:

In this step, we find the new centroid by taking the average of all the points assigned to that cluster.

$$c_i = \frac{1}{|Si|} \sum_{xi \in Si} x_i$$

S_i is the set of all points assigned to the i th cluster

Step 4:

In this step, we repeat step 2 and 3 until none of the cluster assignments change

That means until our clusters remain stable, we repeat the algorithm

Available on notes.arpanadhikari7.com.np

Applications of K-Means Clustering:

- k-means can be applied to data that has a smaller number of dimensions, is numeric, and is continuous. such as document clustering, identifying crime-prone areas, customer segmentation, insurance fraud detection, public transport data analysis, clustering of IT alerts…etc.

notes.arpanadhikari7.com.np

- K-means clustering can be applied to machine learning or data mining.

Available on notes.arpanadhikari7.com.np

- **Used as acoustic data in speech understanding to convert waveforms into one of k categories (known as Vector Quantization or Image Segmentation.**

- **Also used for choosing color palettes on old fashioned graphical display devices and Image Quantization.**

notes.arpanadhikari7.com.np

Introduction to Neuron

A neuron is a cell in brain whose principle function is the collection, Processing, and dissemination of electrical signals. Brains Information processing capacity comes from networks of such neurons. Due to this reason some earliest AI work aimed to create such artificial networks.

A neural network can be defined as a model of reasoning based on the human brain.

The brain consists of a densely interconnected set of nerve cells, or basic information processing units, called neurons

notes.arpanadhikari7.com.np

The human brain incorporates nearly 10 billion neurons and 60 trillion connections, synapses, between them. By using multiple neurons simultaneously, the brain can perform its functions much faster than the fastest computers in existence today.

Each neuron has a very simple structure, but an army of such elements constitutes a tremendous processing power

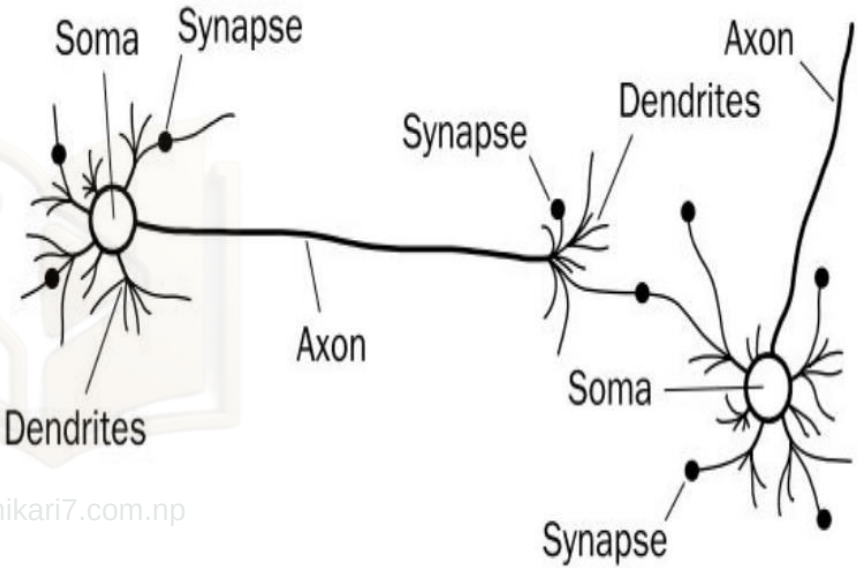
Neuron: fundamental functional unit of all nervous system tissue

Soma: cell body, contain nucleus

Dendrites: a number of fibres, input

Axon: single long fibre with many branches, output

Synapse: junction of dendrites and axon, each neuron form synapse with 10 to 100000 other neurons



notes.arpanadhikari7.com.np

Neural Network

An Artificial Neural Network (ANN) is an information processing paradigm that is inspired by the way biological nervous systems, such as the brain, process information.

The key element of this paradigm is the novel structure of the information processing system.

It is composed of a large number of highly interconnected processing elements (neurones) working in unison to solve specific problems. ANNs, like people, learn by example.

An ANN is configured for a specific application, such as pattern recognition or data classification, through a learning process.

notes.arpanadhikari7.com.np

Why use Neural Network?

Neural networks, with their remarkable ability to derive meaning from complicated or imprecise data, can be used to extract patterns and detect trends that are too complex to be noticed by either humans or other computer techniques.

A trained neural network can be thought of as an "expert" in the category of information it has been given to analyze. Other advantages include:

1. Adaptive learning: An ability to learn how to do tasks based on the data given for training or initial experience.

2. Self-Organisation: An ANN can create its own organisation or representation of the information it receives during learning time.

3. Real Time Operation: ANN computations may be carried out in parallel, and special hardware devices are being designed and manufactured which take advantage of this capability.

4. Fault Tolerance via Redundant Information Coding: Partial destruction of a network leads to the corresponding degradation of performance. However, some network capabilities may be retained even with major network damage

notes.arpanadhikari7.com.np

Neural networks take a different approach to problem solving than that of conventional computers.

Conventional computers use an algorithmic approach i.e. the computer follows a set of instructions in order to solve a problem.

Unless the specific steps that the computer needs to follow are known the computer cannot solve the problem.

That restricts the problem solving capability of conventional computers to problems that we already understand and know how to solve.

But computers would be so much more useful if they could do things that we don't exactly know how to do.

Neural networks process information in a similar way the human brain does.

The network is composed of a large number of highly interconnected processing elements(neurones) working in parallel to solve a specific problem.

notes.arpanadhikari7.com.np

Neural networks learn by example.

They cannot be programmed to perform a specific task.

The examples must be selected carefully otherwise useful time is wasted or even worse the network might be functioning incorrectly.

The disadvantage is that because the network finds out how to solve the problem by itself, its operation can be unpredictable.

On the other hand, conventional computers use a cognitive approach to problem solving; the way the problem is to solved must be known and stated in small unambiguous instructions.

notes.arpanadhikari7.com.np

These instructions are then converted to a high level language program and then into machine code that the computer can understand.
These machines are totally predictable; if anything goes wrong is due to a software or hardware fault.

Available on notes.arpanadhikari7.com.np

Units of Neural Network

Nodes(units): Nodes represent a cell of neural network.

Links: another node.

Activation: Activations are inputs to or outputs from a unit.

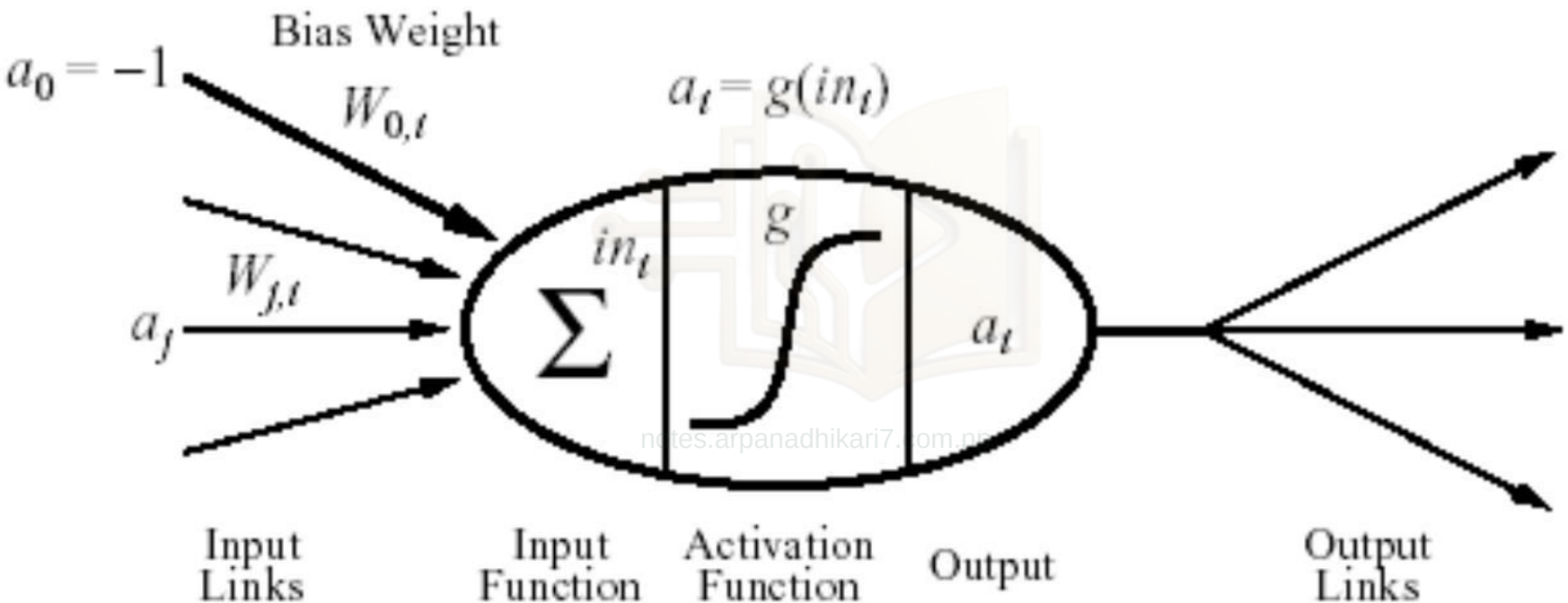
Weight: Each link has weight associated with it which determines strength and sign of the connection.

Activation function: A function which is used to derive output activation from the input activations to a given node is called activation function.

arpanadhikari7.com.np

Bias Weight: Bias weight is used to set the threshold for a unit. Unit is activated when the weighted sum of real inputs exceeds the bias weight.

Simple Neural Network model



notes.arpanadhikari7.com.np

It fires when a linear combination of its inputs exceeds some threshold.

A neural network is composed of nodes (units) connected by directed links A link from unit j to i serve to propagate the activation a_j from j to i. Each link has some numeric weight $W_{j,i}$ associated with it, which determines strength and sign of connection.

Each unit first computes a weighted sum of it's inputs:

$$in_i = \sum_{J=0}^{n} W_{j,i}\ a_j$$

Then it applies activation function g to this sum to derive the output:

$$a_i = g(in_i) = g\left(\sum_{J=0}^{n} W_{j,i}\ a_j\right)$$

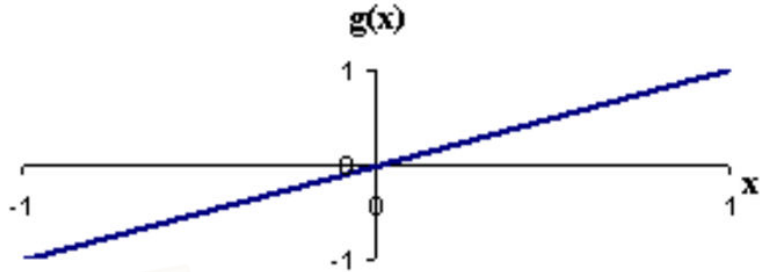
Here, a_j output activation from unit j and $W_{j,i}$ is the weight on the link j to this node.
Activation function typically falls into one of three categories:

- Linear
- Threshold (*Heaviside function)*
- Sigmoid
- Sign

notes.arpanadhikari7.com.np

For **linear activation functions**, the output activity is proportional to the total weighted output.

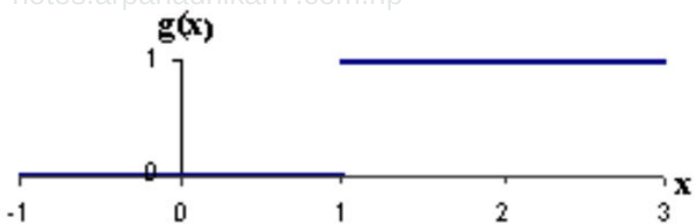
$g(x) = k\,x + c$, where k and x are constant



For **threshold activation functions**, the output are set at one of two levels, depending on whether the total input is greater than or less than some threshold value.

$g(x) = 1$ if $x >= k$

$= 0$ if $x < k$

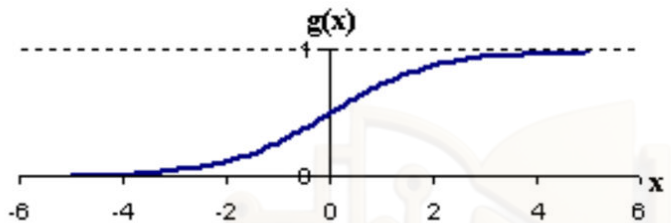


notes.arpanadhikari7.com.np

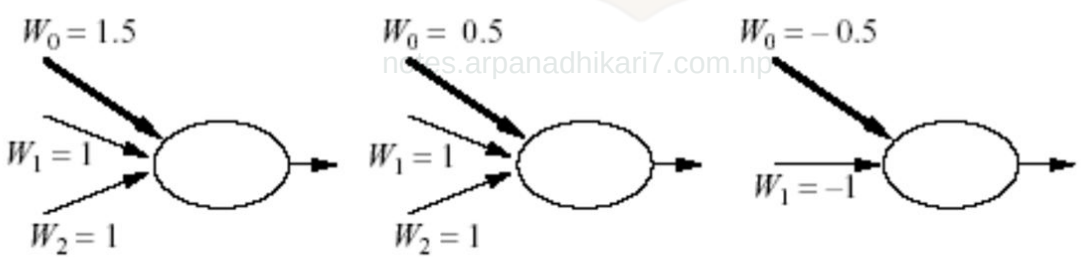
Available on notes.arpanadhikari7.com.np

For **sigmoid activation functions**, the output varies continuously but not linearly as the input changes. Sigmoid units bear a greater resemblance to real neurons than do linear or threshold units. It has the advantage of differentiable.

$g(x) = 1/ (1 + e^{-x})$



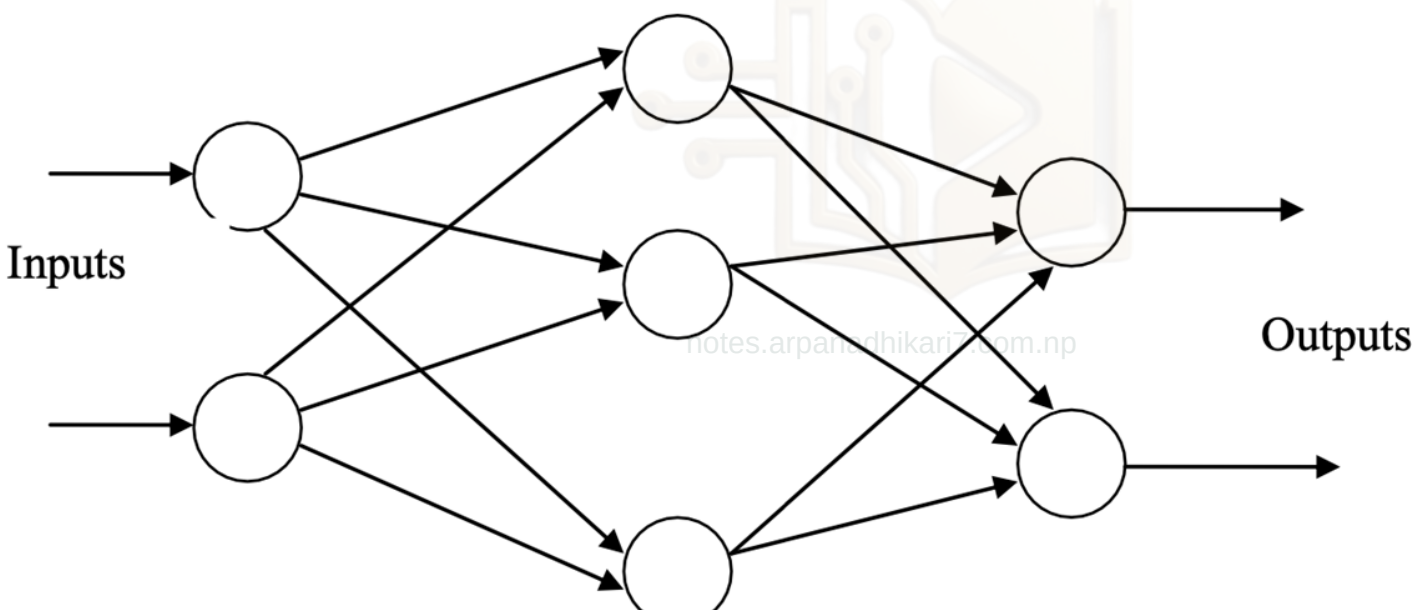
Realizing logic gates by using Neurons:



notes.arpanadhikari7.com.np

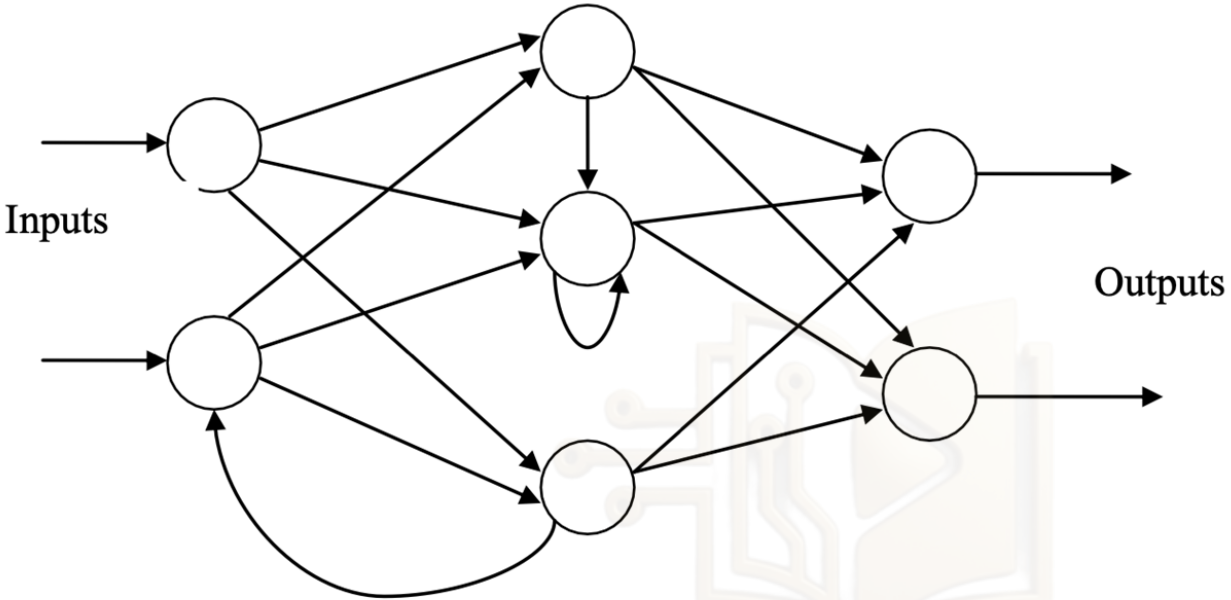
Feed-forward networks:

Feed-forward ANNs allow signals to travel one way only; from input to output. There is no feedback (loops) i.e. the output of any layer does not affect that same layer. Feed-forward ANNs tend to be straight forward networks that associate inputs with outputs. They are extensively used in pattern recognition. This type of organization is also referred to as bottom-up or top-down.



Available on notes.arpanadhikari7.com.np

Feedback networks (Recurrent networks:)



notes.arpanadhikari7.com.np

Feedback networks (figure 1) can have signals traveling in both directions by introducing loops in the network. Feedback networks are very powerful and can get extremely complicated. Feedback networks are dynamic; their 'state' is changing continuously until they reach an equilibrium point. They remain at the equilibrium point until the input changes and a new equilibrium needs to be found. Feedback architectures are also referred to as interactive or recurrent.

Available on notes.arpanadhikari7.com.np

- The neuron computes the weighted sum of the input signals and compares the result with a **threshold value**, θ. If the net input is less than the threshold, the neuron output is −1. But if the net input is greater than or equal to the threshold, the neuron becomes activated and its output attains a value +1.
- The neuron uses the following transfer or **activation function**
- $X = \sum_{i=1}^{n} x_i w_i \qquad Y = \begin{cases} +1 \; if \; X \geq \theta \\ -1 \; if \; X \leq \theta \end{cases}$
- This type of activation function is called a **sign function**

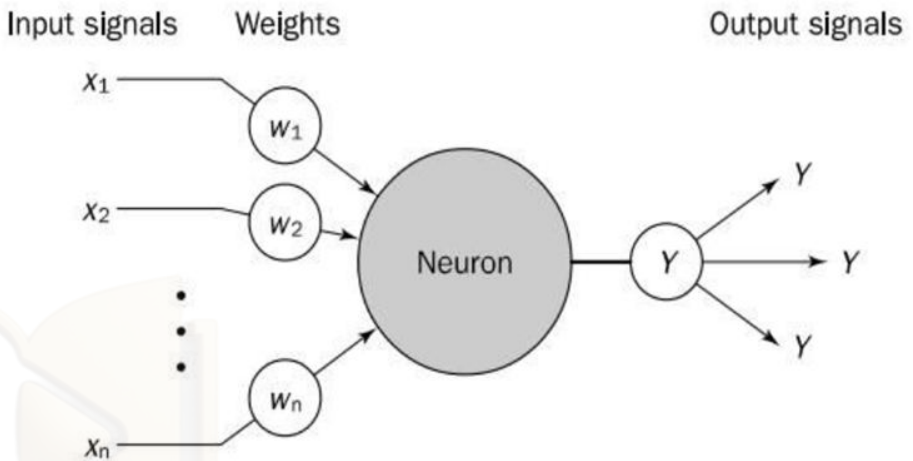


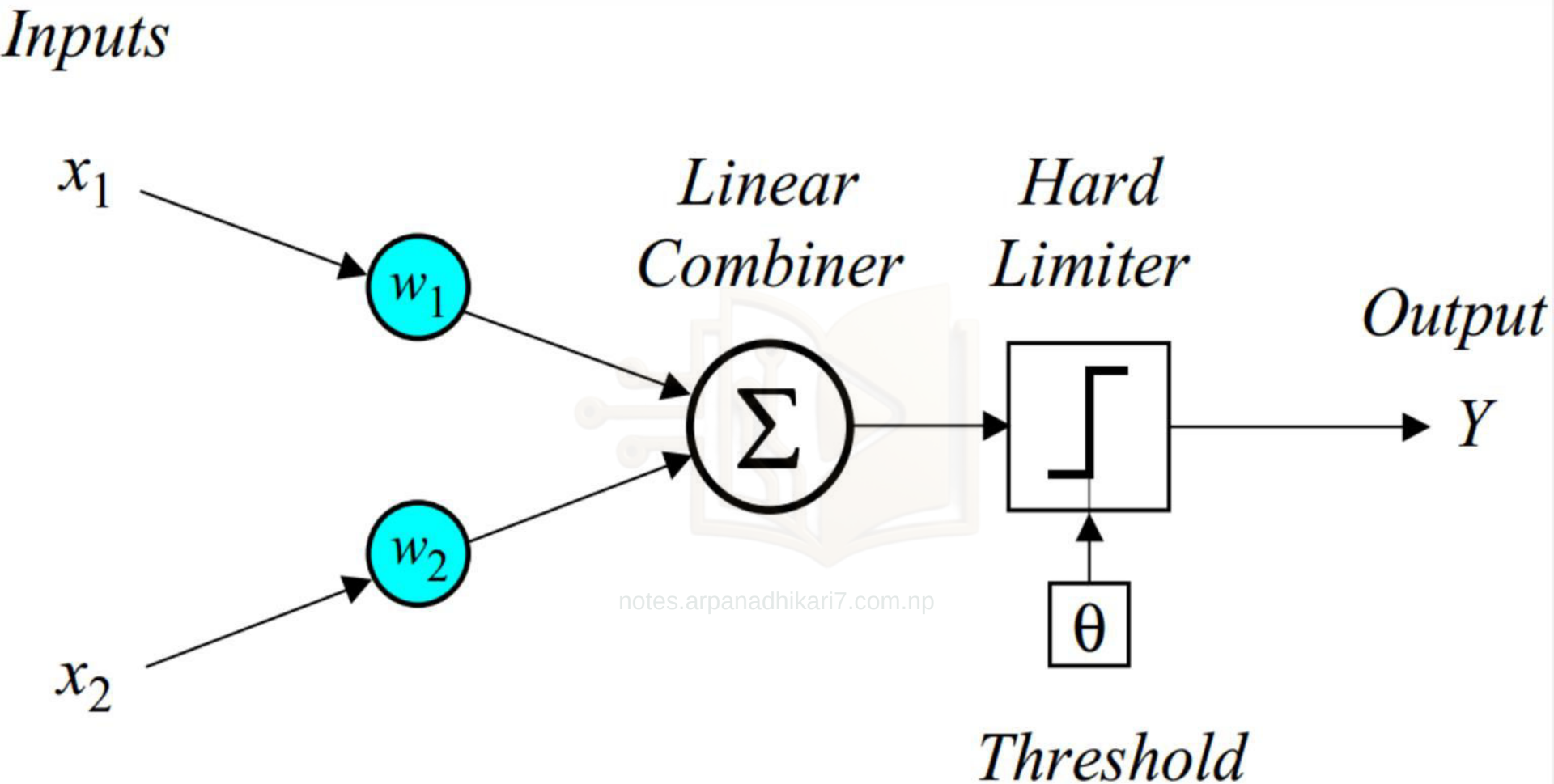
Diagram of a neuron

notes.arpanadhikari7.com.np

Perceptron

- In 1958, Frank Rosenblatt introduced a training algorithm that provided the first procedure for training a simple ANN : a **perceptron**
- The perceptron is the simplest form of a neural network. It consists of a single neuron with *adjustable* synaptic weights and a **hard limiter**
- The operation of Rosenblatt's perceptron is based on the McCulloch and Pitts neuron model. The model consists of a **linear combiner** followed by a hard limiter
- The weighted sum of the inputs is applied to the hard limiter, which produces an output equal to +1 if its input is positive and -1 if its input is negative

Available on notes.arpanadhikari7.com.np

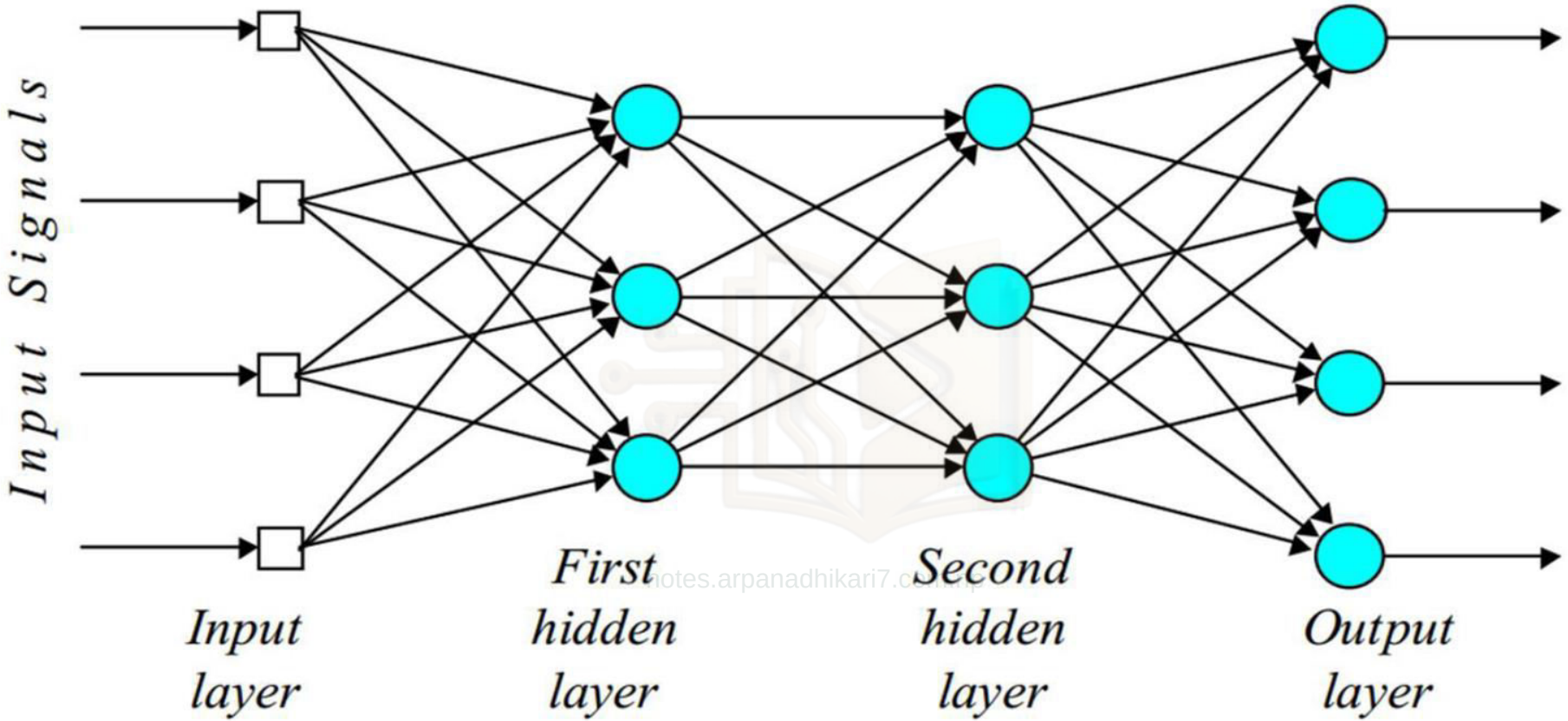


notes.arpanadhikari7.com.np

Multilayer perceptron

- A multi layer perceptron is a feed forward neural network with one or more hidden layers
- The network consists of :
  - Input Layer
  - Hidden Layer
  - Output Layer
- The input signal is propagated in a forward direction in a layer-by-layer basis

notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

Back Propagation

Learning in a multilayer network proceeds the same way as for a perceptron

A training set of input patterns is presented to the network

The network computes its output pattern, and if there is an error –or other word difference between actual and desired output pattern – the weight are adjusted to reduce the error

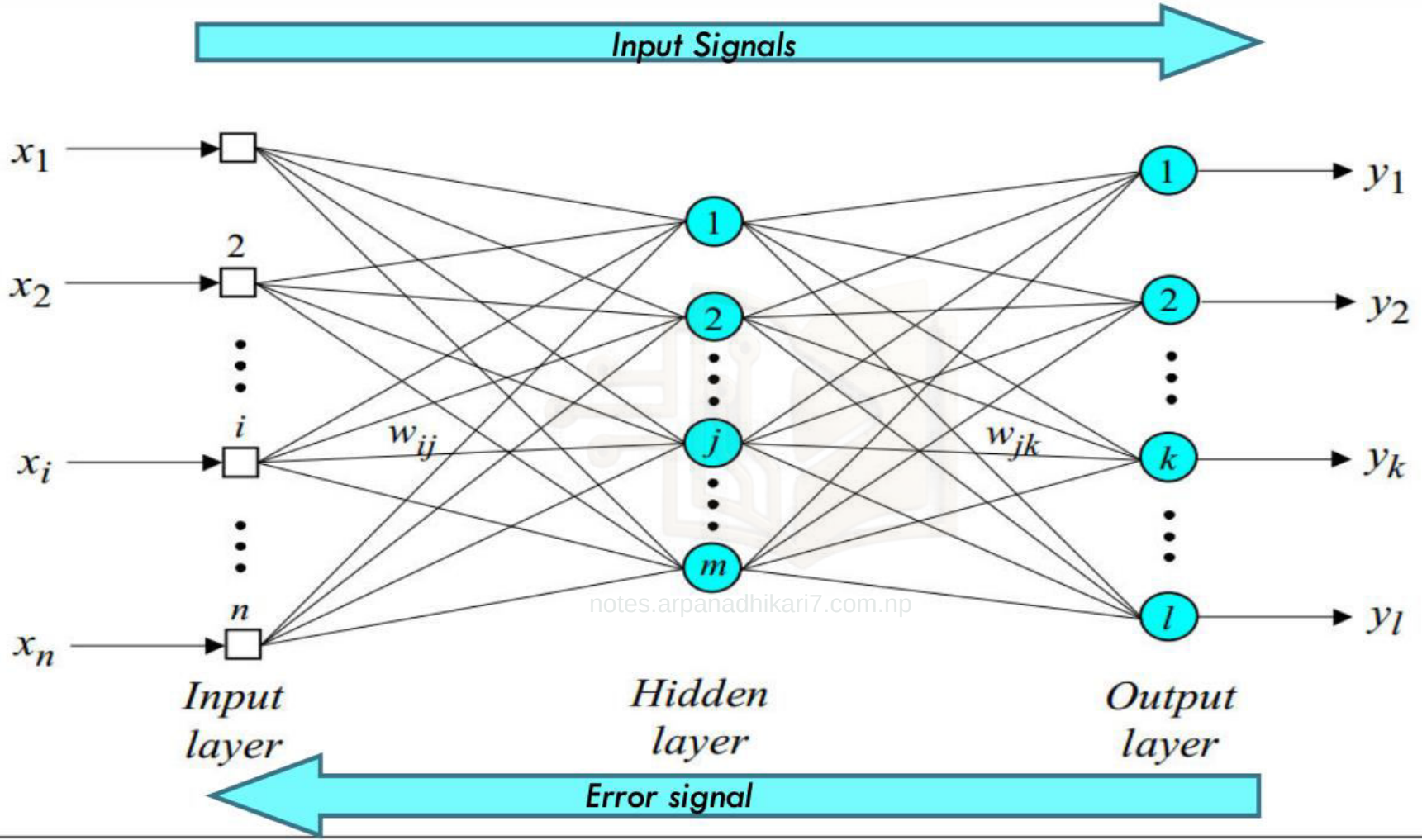
In a back-propagation neural network, the learning algorithm has two phases

notes.arpanadhikari7.com.np

First, a training input pattern is presented to the network input layer. The network propagates the input pattern from layer to layer until the output pattern is generated by the out layer

If this pattern is different from the desired output, an error is calculated and then propagated backwards through the network from the output layer to the input layer. The weights are modified as

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Example of Neural Network

```
import numpy

#Sigmoid Function: Activation function

def func(x,deriv=False):

    if(deriv==True):

        return x*(1-x)

    return 1/(1+np.exp(-x))
```

notes.arpanadhikari7.com.np

Example of Neural Network

```
X=np.array([
    [0,0,1],[0,1,1],[1,0,1],[1,1,1]
])
y=np.array([0,0,1,1]).T
```


notes.arpanadhikari7.com.np

Example of Neural Network

```
w0=np.random.random((3,1))


for i in range(1000):
    I0= X
    I1=np.dot(I0,w0)
    I1=func(I1)
    error=y-I1
    delta_I1 = error*func(I1,True)
    w0 = w0+ np.dot(I0.T,delta_I1)
print("The output is:\n",I1)
```

notes.arpanadhikari7.com.np

Linear Regression

```
import numpy as np

from sklearn.linear_model import LinearRegression

x = np.array([5, 15, 25, 35, 45, 55]).reshape((-1, 1))

y = np.array([5, 20, 14, 32, 22, 38])

model = LinearRegression()

model.fit(x, y)
```

notes.arpanadhikari7.com.np

Linear Regression

```
model = LinearRegression().fit(x, y)

r_sq = model.score(x, y)

print(f"coefficient of determination: {r_sq}")

new_model = LinearRegression().fit(x, y.reshape((-1, 1)))

print(f"intercept: {new_model.intercept_}")

print(f"slope: {new_model.coef_}")

y_pred = model.predict(x)

print(f"predicted response:\n{y_pred}")
```

notes.arpanadhikari7.com.np

Same model reflected from equation

```
y_pred = model.intercept_ + model.coef_ * x

print(f"predicted response:\n{y_pred}")

#New inputs

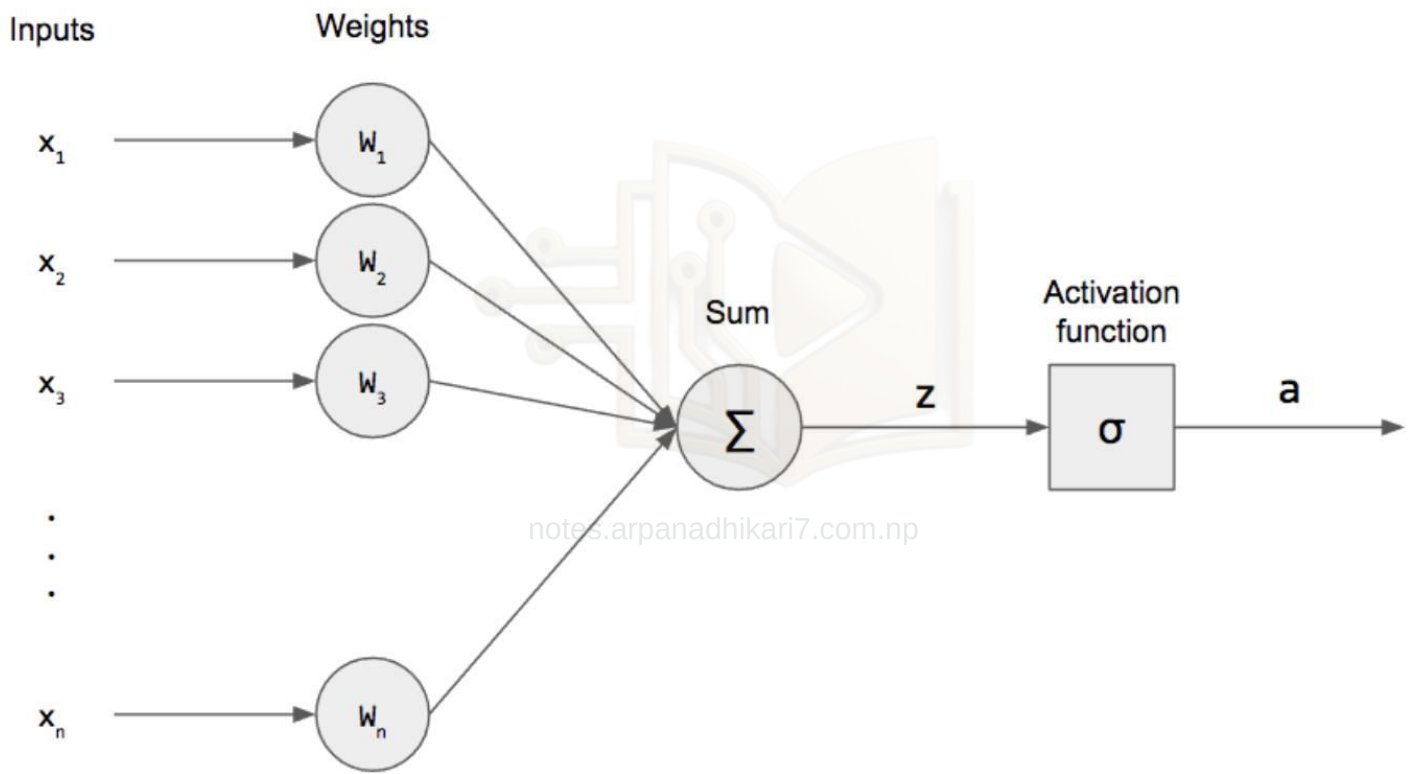
x_new = np.arange(5).reshape((-1, 1))

y_new = model.predict(x_new)

print(y_new)
```

notes.arpanadhikari7.com.np

Gate realization using neural network



notes.arpanadhikari7.com.np

Steps to show how neural network learn

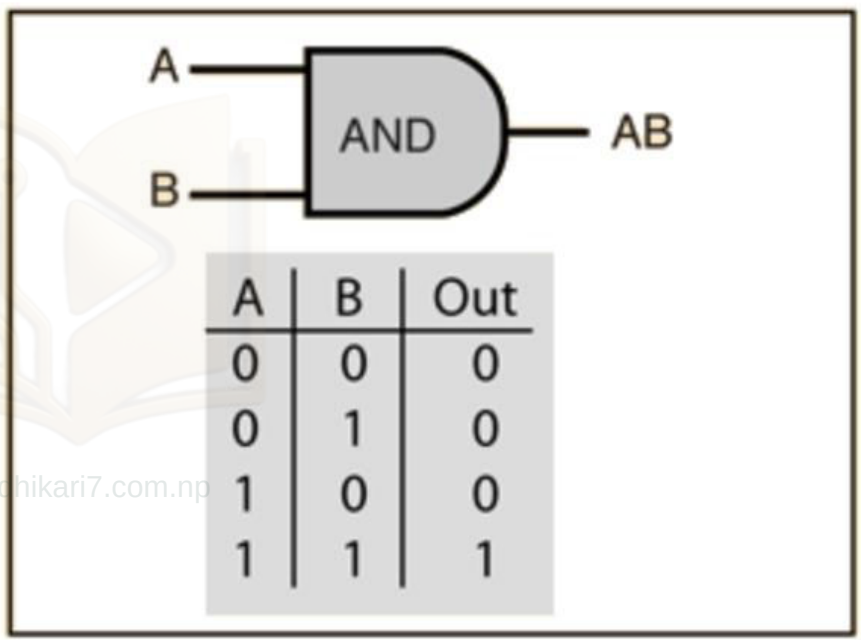
- Initialize weight values and bias
- Forward Propagate
- Check the error
- Backpropagate and Adjust weights and bias
- Repeat for all training examples

notes.arpanadhikari7.com.np

AND gate

The question is, what are the weights and bias for the AND perceptron?

First, we need to understand that the output of an AND gate is 1 only if both inputs (in this case, x1 and x2) are 1. So, following the steps listed above;



A	B	Out
0	0	0
0	1	0
1	0	0
1	1	1

AND Gate

notes.arpanadhikari7.com.np

Row 1

- From w1*x1+w2*x2+b, initializing w1, w2, as 1 and b as –1, we get;

x1(1)+x2(1)–1

- Passing the first row of the AND logic table (x1=0, x2=0), we get;

0+0–1 = –1

notes.arpanadhikari7.com.np

- From the Perceptron rule, if Wx+b≤0, then y`=0. Therefore, this row is correct, and no need for Backpropagation.

Row 2

- Passing (x1=0 and x2=1), we get;

$0+1-1 = 0$

- From the Perceptron rule, if $Wx+b \leq 0$, then $y` = 0$. This row is correct, as the output is 0 for the AND gate.
- From the Perceptron rule, this works (for both row 1, row 2 and 3).

notes.arpanadhikari7.com.np

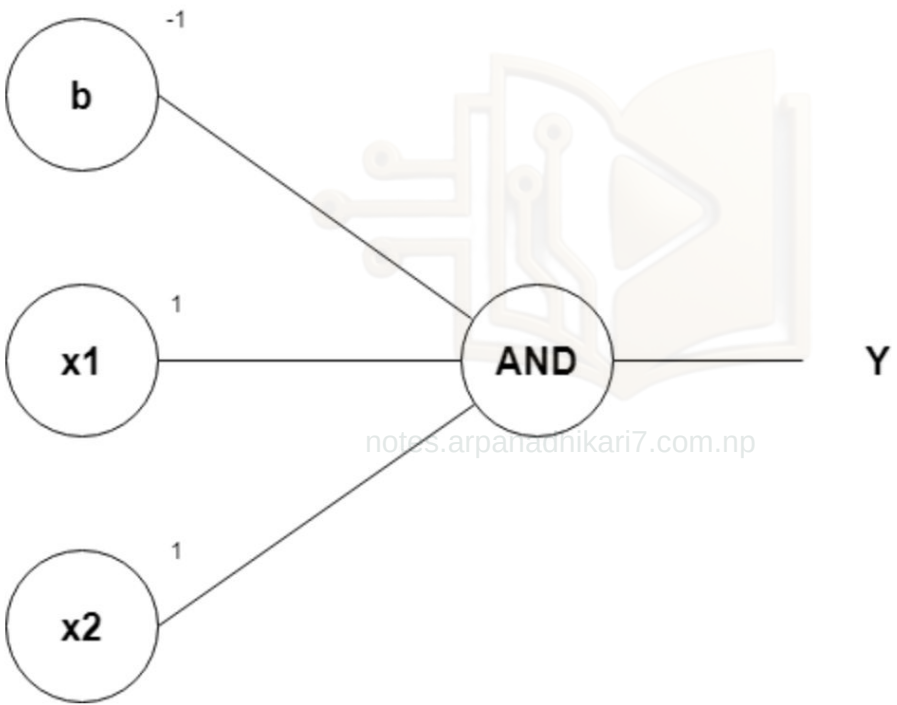
Row 4

- Passing (x1=1 and x2=1), we get;

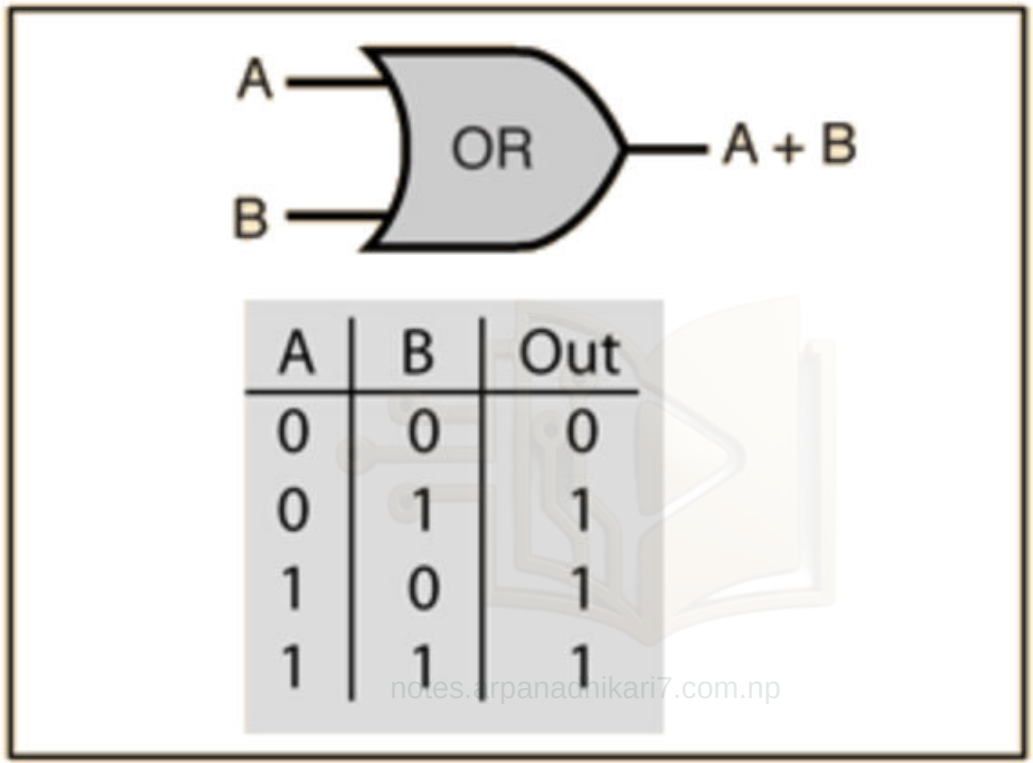
$1+1-1 = 1$

Therefore, we can conclude that the model to achieve an AND gate, using the Perceptron algorithm is;

$x1+x2-1$



notes.arpanadhikari7.com.np



A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1

notes.arpanadhikari7.com.np

OR Gate

From the diagram, the OR gate is 0 only if both inputs are 0.

Row 1

- From w1x1+w2x2+b, initializing w1, w2, as 1 and b as –1, we get;

x1(1)+x2(1)–1

- Passing the first row of the OR logic table (x1=0, x2=0), we get;

notes.arpanadhikari7.com.np

0+0–1 = –1

- From the Perceptron rule, if Wx+b≤0, then y`=0. Therefore,

Row 2

- Passing (x1=0 and x2=1), we get;

$0+1-1 = 0$

- From the Perceptron rule, if Wx+b <= 0, then y`=0. Therefore, this row is incorrect.
- So we want values that will make inputs x1=0 and x2=1 give y` a value of 1. If we change w2 to 2, we have;

notes.arpanadhikari7.com.np

$0+2-1 = 1$

- From the Perceptron rule, this is correct for both the row 1 and 2.

Row 3

- Passing (x1=1 and x2=0), we get;

$1+0-1 = 0$

- From the Perceptron rule, if Wx+b <= 0, then y`=0. Therefore, this row is incorrect.
- Since it is similar to that of row 2, we can just change w1 to 2, we have;

notes.arpanadhikari7.com.np

$2+0-1 = 1$

- From the Perceptron rule, this is correct for both the row 1, 2 and 3.

Row 4

- Passing (x1=1 and x2=1), we get;

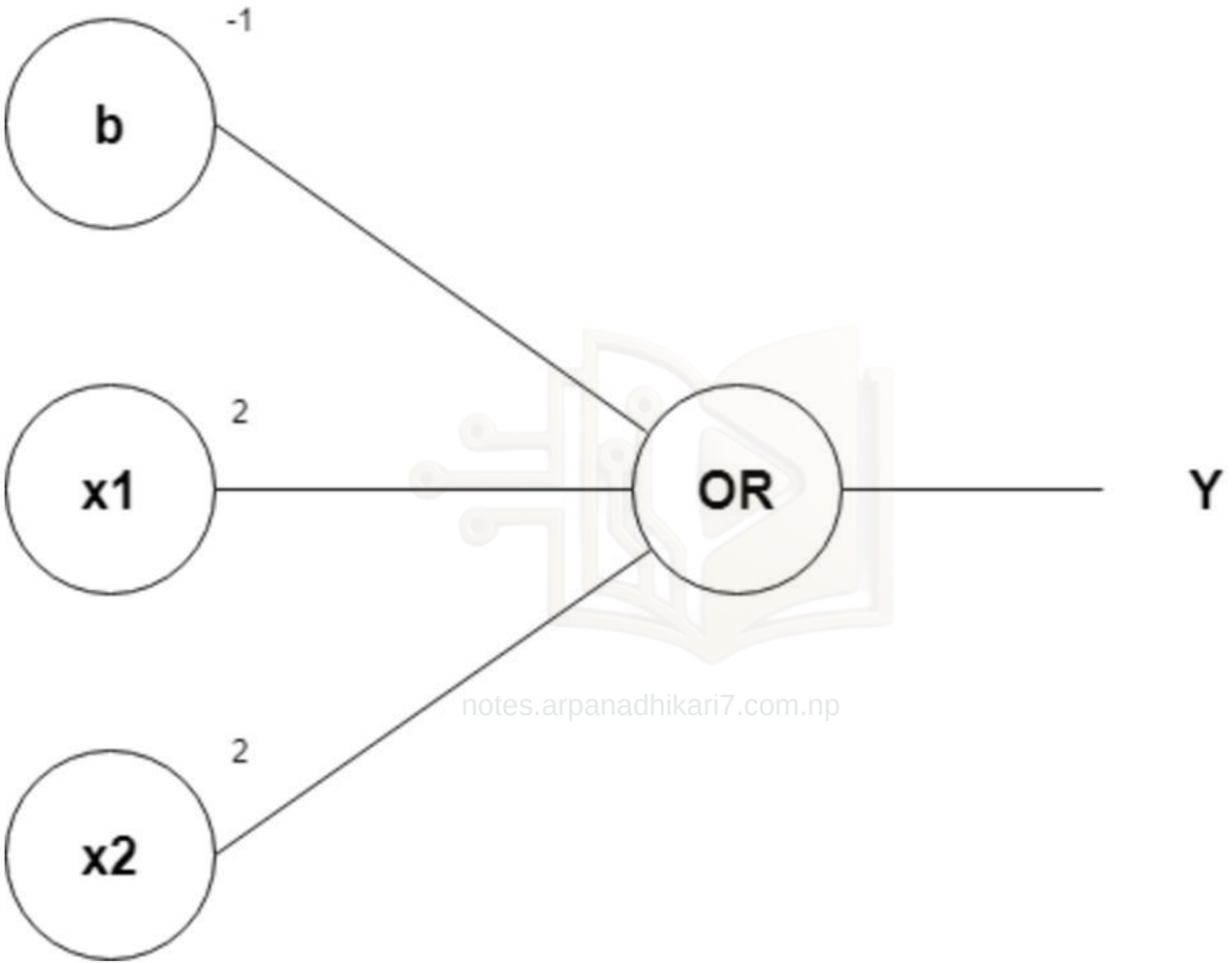
$2+2-1 = 3$

- Again, from the perceptron rule, this is still valid. Quite Easy!

Therefore, we can conclude that the model to achieve an OR gate, using the Perceptron algorithm is;

notes.arpanadhikari7.com.np

$2x1+2x2-1$



notes.arpanadhikari7.com.np

Keras and TensorFlow

We need to analyze the libraries

```
import numpy as np

from keras.models import Sequential

from keras.layers.core import Dense
```

notes.arpanadhikari7.com.np

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense
```



Input and Output Data

```
training_data = np.array([[0,0],[0,1],[1,0],[1,1]],
"float32")

 target_data = np.array([[0],[0],[0],[1]], "float32")
```

Since we are training the model to examine or model the AND gate

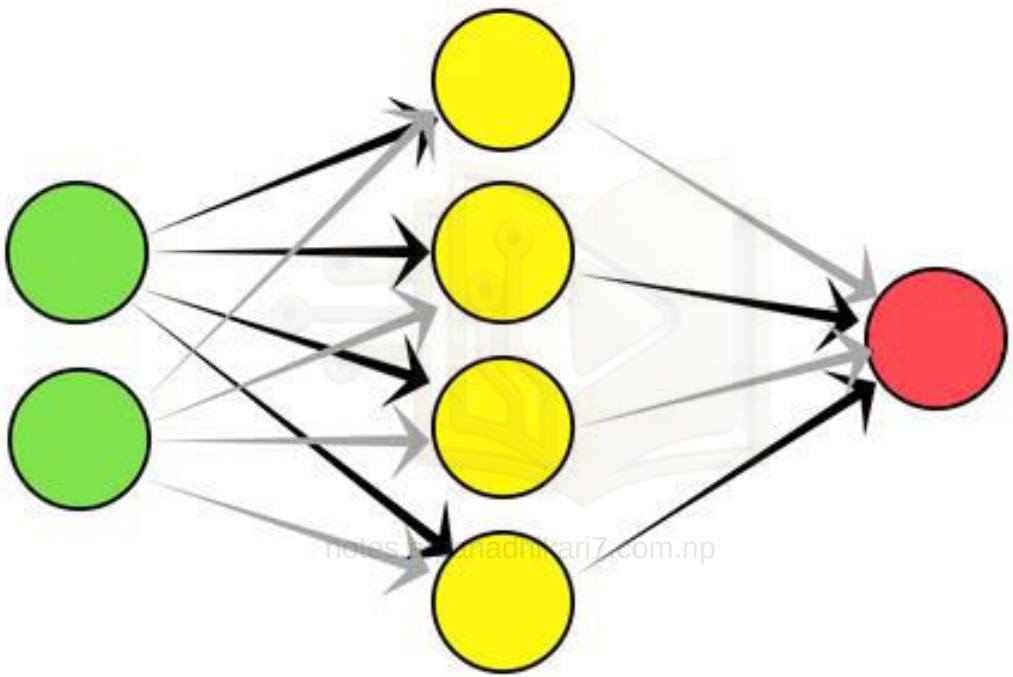
notes.arpanadhikari7.com.np

```
model = Sequential()

model.add(Dense(16,
input_dim=2,activation='relu'))

model.add(Dense(1, activation='sigmoid'))
```

notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

```
model.compile(loss='mean_squared_error',optimizer='adam
',metrics=['binary_accuracy'])
```

```
model.fit(training_data, target_data, epochs=1000)
```

notes.arpanadhikari7.com.np

Training results

Let's check the training outputs

```
Epoch 1/10004/4 [==============================] - 0s 43ms/step - loss: 0.2634 - binary_accuracy: 0.5000
Epoch 2/10004/4 [==============================] - 0s 457us/step - loss: 0.2630 - binary_accuracy: 0.2500
```

We can see it was kind of luck the firsts iterations and accurate for half of the outputs, but after the second it only provides a correct result of one-quarter of the iterations. Then, in the 24th epoch recovers 50% of accurate results, and this time is not a coincidence, is because it correctly adjusted the network's weights.

```
Epoch 24/1000 4/4 [==============================] - 0s 482us/step - loss: 0.2549 - binary_accuracy: 0.5000
Epoch 107/1000 4/4 [==============================] - 0s 621us/step - loss: 0.2319 - binary_accuracy: 0.7500
Epoch 169/1000 4/4 [==============================] - 0s 1ms/step - loss: 0.2142 - binary_accuracy: 1.0000
```

Evaluation and prediction

```
scores = model.evaluate(training_data,
target_data)

print("\n%s: %.2f%%" % (model.metrics_names[1],
scores[1]*100))
print (model.predict(training_data).round())
```

notes.arpanadhikari7.com.np

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.utils import to_categorical
```

notes.arpanadhikari7.com.np

```
# Create the dataset
data = {
    'Age': [36, 42, 23, 52, 43, 44, 66, 35, 52, 35, 24, 18, 45, 41],
    'Experience': [10, 12, 4, 4, 21, 14, 3, 14, 13, 5, 3, 3, 9, 5],
    'Rank': [9, 4, 6, 4, 8, 5, 7, 9, 7, 9, 5, 7, 9, 4],
    'Nationality': ['UK', 'USA', 'N', 'USA', 'USA', 'UK', 'N', 'UK', 'N', 'N', 'USA', 'UK', 'UK', 'N'],
    'Go': ['NO', 'NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'YES', 'YES', 'YES', 'YES', 'YES', 'YES', 'NO']
}
```

notes.arpanadhikari7.com.np

```
# Create DataFrame
df = pd.DataFrame(data)


# Encode categorical variables
label_encoders = {}
for column in ['Nationality', 'Go']:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le
```

notes.arpanadhikari7.com.np

```
# Define features and target variable
X = df.drop('Go', axis=1)
y = df['Go']


# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

notes.arpanadhikari7.com.np

```
# Convert target variable to categorical
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
print(y_train)


# Create the neural network model
model = Sequential()
model.add(Dense(8, input_dim=X_train.shape[1], activation='relu'))  # Input layer
model.add(Dense(8, activation='tanh'))  # Hidden layer
model.add(Dense(2, activation='sigmoid'))  # Output layer (2 classes)
```

notes.arpanadhikari7.com.np

```
# Compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=100, batch_size=5, verbose=1)
```

notes.arpanadhikari7.com.np

```
# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {accuracy:.2f}')

# Make predictions
predictions = model.predict(X_test)
predicted_classes = predictions.argmax(axis=1)
```

notes.arpanadhikari7.com.np

```
# Convert predicted classes back to original labels
predicted_labels = label_encoders['Go'].inverse_transform(predicted_classes)
print(f'Predicted labels: {predicted_labels}')

from sklearn.metrics import confusion_matrix, classification_report

# Print confusion matrix and classification report
print(confusion_matrix(y_test.argmax(axis=1), predicted_classes))
print(classification_report(y_test.argmax(axis=1), predicted_classes))
```

Available on notes.arpanadhikari7.com.np